

Introduction to Quantum Computing: Quantum Algorithms and Qiskit

NPTEL • IIT Madras • IBM Research — Complete Learning Handbook

Instructors: Prof. Prabha Mandayam (IIT Madras) • Dr. Anupama Ray (IBM Research) • Dr. Sheshashayee Raghunathan (IBM Systems)

About This Playlist

Attribute	Detail
What it teaches	Foundations of quantum computing: from qubits and gates to quantum algorithms (Deutsch-Jozsa, Grover's), hands-on IBM Qiskit programming, quantum error correction, and NISQ-era algorithms (VQE, QAOA)
Who it is for	Students, researchers, and faculty from Computer Science, Physics, Mathematics, and Chemistry with curiosity about the next computing paradigm
Prerequisites	Basic linear algebra (vectors, matrices, eigenvalues); introductory Python helpful but not required
Estimated study time	~30–40 hours (12 hours of video + exercises + hands-on Qiskit labs)
Technologies used	IBM Qiskit (Python), IBM Quantum Experience (cloud), IBM Circuit Composer
Industry support	IBM, Microsoft, Intel, Google, D-Wave, IonQ, Honeywell, TCS, Infosys
Final skills gained	Represent qubits mathematically, design quantum circuits, implement and run quantum algorithms on real IBM hardware, understand quantum error correction, and apply NISQ algorithms to optimization

Learning Roadmap

Why Quantum Computing Exists

Classical computers process information as **bits** — each a 0 or 1. Every transistor in your CPU enforces a binary choice. For most everyday tasks this works perfectly. But a set of computationally hard problems exist — factoring enormous integers, simulating quantum chemistry, searching unsorted databases — for which the best classical algorithms would require longer than the age of the universe even on the fastest supercomputers imaginable.

Nature itself computes with quantum mechanics. Electrons in molecules don't pick one energy state; they exist in quantum superpositions simultaneously. If we could build a computer that natively uses quantum rules, it could *simulate* nature in polynomial time, not exponential time.

Quantum computing is not simply "faster computing." It is a **fundamentally different model of computation** exploiting two phenomena classical physics forbids: **superposition** (being in multiple states at once) and **entanglement** (non-local correlations between particles).

Why It Is Important Right Now

- **Cryptography threat:** Shor's algorithm (quantum) can factor large numbers exponentially faster than any known classical algorithm, breaking RSA encryption. Governments worldwide are already transitioning to post-quantum cryptography.
- **Drug discovery:** Simulating molecular interactions at quantum scale could discover drugs in days instead of decades.
- **Optimization:** Logistics, finance, and supply-chain optimization involve combinatorial search spaces too large for classical exhaustive search.
- **Machine learning:** Quantum machine learning may offer speedups for certain classification and kernel tasks.
- **National priority:** India's National Mission on Quantum Technologies and Applications (NM-QTA) has allocated ₹8000 crore; IIT Madras was the first Indian institution to join the IBM Quantum Network.

Where It Fits in Real Software Engineering

```
Problem Domain (Chemistry / Cryptography / Optimization)
  ↓
Quantum Algorithm Design (Grover, Shor, VQE, QAOA...)
  ↓
Quantum Circuit (gates acting on qubits)
  ↓
Qiskit / Quantum SDK (Python abstraction layer)
  ↓
IBM Quantum Composer (visual circuit builder)
  ↓
Transpiler (maps circuit to native hardware gates)
  ↓
IBM Quantum Hardware (superconducting qubits, real device)
  ↓
Measurement → Classical post-processing → Result
```

Roadmap Across This Course

WEEK 0 – LINEAR ALGEBRA BRUSHUP (prerequisite)

↓

WEEK 1 – FOUNDATION

- Quantum computing motivation & roadmap
- Quantum mission in India
- Applications landscape
- Quantum basics: qubits, superposition, entanglement

↓

WEEK 2 – TOOLS & HARDWARE

- IBM Quantum Composer (visual)
- IBM Quantum Lab with Qiskit (programmatic)
- Quantum gates: single-qubit and two-qubit
- Bell states, no-cloning theorem, teleportation

↓

WEEK 3 – ALGORITHMS I & II

- Oracle-based computation
- Deutsch-Jozsa algorithm
- Grover's search algorithm
- Hands-on Qiskit demonstrations

↓

WEEK 4 – ERROR CORRECTION & NISQ ERA

- Quantum error correction theory
- NISQ devices and their limitations
- Variational Quantum Eigensolver (VQE)
- Quantum Approximate Optimization Algorithm (QAOA)
- Industrial applications

WEEK 1 — FOUNDATION

Lecture 1 — Quantum Computing Roadmap

Objective

Orient the learner to the entire quantum computing landscape: what quantum computing is, why it matters, where it is today, and what the course will teach. This lecture is the "map" before the territory.

Big Picture

Classical Bit (0 or 1)



Transistor → Logic Gate → CPU → Program → Output

Quantum Bit (superposition of 0 AND 1)



Quantum Gate → Quantum Circuit → Quantum Algorithm → Measurement → Classical Output

The classical and quantum stacks are analogous but the physics underneath is completely different. Classical bits are implemented by whether a transistor is on (1) or off (0). A quantum bit (qubit) is implemented by a physical two-level quantum system — a photon's polarization, the spin of an electron, or the energy level of a superconducting circuit cooled to near absolute zero.

Core Concepts

1. Classical Computing Limitations

Classical computers are built on the **Church-Turing thesis**: anything computable can be computed by a Turing machine in polynomial time if and only if a physical computer can do it in polynomial time. Some problems are in **complexity class P** (efficiently solvable) and others are in **NP** (efficiently verifiable but not efficiently solvable by any known algorithm).

Quantum computing does not solve all hard problems. It provides **superpolynomial speedups for specific problem classes**:

Problem	Classical Best	Quantum Algorithm	Speedup
Unstructured search	$O(N)$	Grover's: $O(\sqrt{N})$	Quadratic
Integer factorization	Sub-exponential	Shor's: $O((\log N)^3)$	Exponential
Quantum simulation	Exponential	Polynomial	Exponential
Linear systems	$O(N^2 \cdot 3)$	HHL: $O(\log N)$	Exponential (conditions apply)

2. Moore's Law and Its Limits

Gordon Moore observed in 1965 that the number of transistors on a chip doubles every ~18 months. This trend held for 50 years, but physical limits are now asserting themselves:

- Transistors are approaching the size of atoms (~5nm today)
- Quantum tunneling at sub-2nm scales causes current leakage
- Heat dissipation becomes unmanageable
- Classical parallelism (more cores) yields diminishing returns for serial algorithms

Quantum computing does not simply "continue Moore's Law." It is a paradigm shift.

3. The Quantum Computing Stack

Every technology has layers. Understanding the quantum stack prevents confusion:

APPLICATIONS	Drug discovery, finance, logistics
ALGORITHMS	Grover, Shor, VQE, QAOA
QUANTUM CIRCUIT MODEL	Gates, qubits, measurement
PROGRAMMING SDK	Qiskit, Cirq, Q#, PennyLane
COMPILER / TRANSPILER	Maps gates to native hardware ops
QUANTUM HARDWARE	Superconducting, ion trap, photonic
PHYSICS	Quantum mechanics, decoherence

4. IBM's Role: From Theory to Hardware

IBM Research has been building quantum computers since 2016 when they put the first quantum computer on the cloud (IBM Quantum Experience). Today IBM operates 100+ qubit devices and provides free cloud access to quantum processors. IBM's Qiskit is the world's most widely used quantum software development kit, with millions of users.

5. Today's Era: NISQ

We are currently in the **Noisy Intermediate-Scale Quantum (NISQ)** era, a term coined by physicist John Preskill in 2018:

- **Noisy:** Qubits lose their quantum state quickly (decoherence). Every gate introduces errors.
- **Intermediate-Scale:** Current devices have 50–1000 physical qubits, not the millions of fault-tolerant qubits needed for large-scale Shor's algorithm.

This course teaches both foundational quantum computing AND NISQ-era algorithms designed to extract value from today's imperfect machines.

Detailed Explanation

What a Qubit Is (Conceptually)

A **qubit** (quantum bit) is the fundamental unit of quantum information. Like a classical bit, it has two basis states: $|0\rangle$ and $|1\rangle$ (read "ket zero" and "ket one" — this is Dirac's bra-ket notation). Unlike a classical bit, a qubit can exist in a **superposition**:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$

Where α and β are **complex numbers** (amplitudes) satisfying $|\alpha|^2 + |\beta|^2 = 1$. The probability of measuring 0 is $|\alpha|^2$ and measuring 1 is $|\beta|^2$.

Critical insight: The qubit is *not* secretly one value that we don't know. Before measurement, the qubit genuinely has no definite value. This is not a philosophical claim but an experimentally verified fact (via Bell inequalities). This is radically different from a classical random variable.

What Entanglement Is (Conceptually)

Two or more qubits can be **entangled**: their states become correlated in a way that cannot be described by any combination of independent single-qubit states. Measuring one qubit instantaneously tells you about the other, regardless of distance. Einstein called this "spooky action at a distance" but experiments confirm it is real.

Entanglement is a resource — it is what gives quantum computers much of their power.

Internal Working

How IBM's Superconducting Qubits Work

IBM implements qubits using **superconducting Josephson junctions** cooled to ~15 millikelvin (colder than outer space):

```
Room Temperature (23°C)
    ↓
Dilution Refrigerator
    ↓
~15 millikelvin
    ↓
Superconducting qubit chip
(Josephson junction = two superconductors
separated by a thin insulating barrier)
```

The qubit is the two lowest energy levels of a nonlinear superconducting resonator. A **microwave pulse** at exactly the qubit's resonant frequency (around 5 GHz) rotates the qubit state — this is how quantum gates are physically applied.

Decoherence: Interaction with the environment (thermal noise, electromagnetic interference) destroys the quantum state. Current IBM qubits have coherence times of ~100–500 microseconds — that is the

window in which all gates and measurements must complete.

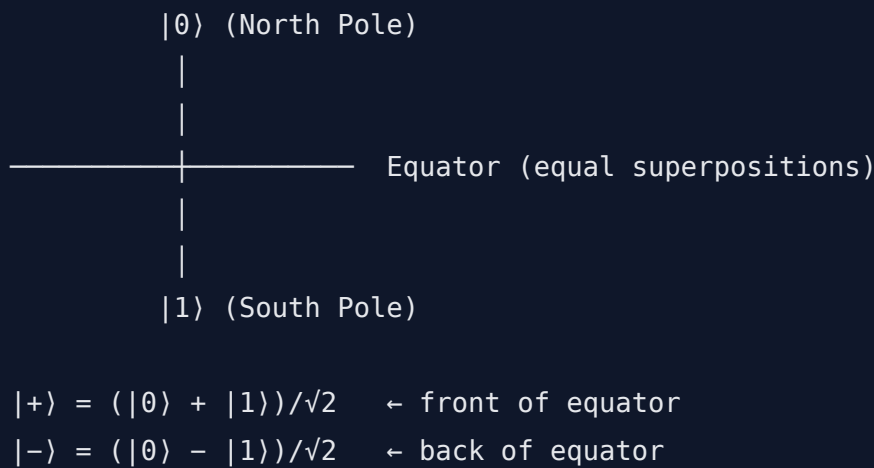
Step-by-Step Execution: Running a Quantum Program

```
1. Researcher designs quantum algorithm
  ↓
2. Writes circuit in Qiskit (Python)
  ↓
3. Transpiler converts to native gate set
  (IBM native gates: CNOT, ID, RZ, SX, X)
  ↓
4. Circuit uploaded to IBM Quantum via cloud API
  ↓
5. Queued on real hardware
  ↓
6. Microwave pulses applied to superconducting qubits
  at 15 millikelvin
  ↓
7. Measurement: qubit state read via microwave resonator
  (quantum → classical signal)
  ↓
8. Result transmitted back to cloud
  ↓
9. Researcher receives bit-string probability distribution
  ↓
10. Classical post-processing extracts answer
```

Visual Mental Models

The Bloch Sphere (Single Qubit)

Every pure single-qubit state can be visualized as a point on a unit sphere called the **Bloch sphere**:



Classical bits can only be at the North or South Pole. Qubits can be anywhere on the surface.

Quantum Advantage Landscape

Problems where quantum helps:

Factoring large numbers
Searching unsorted data
Simulating quantum chemistry
Certain optimization tasks
Breaking RSA encryption

Problems where quantum doesn't help:

Sorting a list
Watching a video
Word processing
Most everyday computing
General NP-hard problems

Real World Usage

- **IBM:** IBM Eagle (127 qubits), IBM Osprey (433 qubits), IBM Condor (1121 qubits) — milestone hardware roadmap
- **Google:** Sycamore processor achieved "quantum supremacy" in 2019 (specific random circuit sampling task)
- **IonQ:** Ion-trap quantum computers used by US Air Force, Goldman Sachs
- **D-Wave:** Quantum annealer used by Volkswagen for traffic optimization, Lockheed Martin for aerospace
- **Quantinuum (Honeywell):** Trapped-ion quantum computers for drug simulation
- **Government:** US NIST finalizing post-quantum cryptographic standards; India's NM-QTA funding ₹8000 crore

Interview Questions

Basic:

- What is the difference between a classical bit and a qubit?

- What is quantum superposition?
- Why is quantum entanglement different from classical correlation?

Intermediate:

- Why can't we just use a very powerful classical computer instead of quantum?
- What problems does quantum computing NOT speed up?
- What is the NISQ era and what are its challenges?

Advanced:

- Explain the relationship between quantum complexity classes BQP and NP.
- Why does Grover's algorithm provide only a quadratic speedup while Shor's provides exponential?

Key Takeaways

- Quantum computing exploits superposition and entanglement for fundamentally new computation
 - NOT a replacement for classical computing — a complement for specific hard problems
 - IBM is the leading enabler of cloud-accessible quantum hardware
 - We are in the NISQ era: noisy, small-scale, but already useful
 - The quantum stack mirrors classical computing from physics to application
-

Lecture 2 — Quantum Mission in India

Objective

Provide context for India's national quantum strategy, the role of IIT Madras as IBM's academic partner, and the opportunity landscape for Indian professionals entering quantum computing.

Core Concepts

India's National Mission on Quantum Technologies and Applications (NM-QTA)

Launched in Budget 2020-21 with an outlay of **₹8000 crore (approximately \$1 billion USD)** over five years, NM-QTA covers:

- Quantum computing hardware development
- Quantum communication and cryptography
- Quantum sensing and metrology
- Quantum materials and devices
- Human resource development

The mission recognizes that quantum technology will be as strategically important as nuclear and space technology.

IIT Madras as IBM Quantum Network Partner

IIT Madras was the **first institution in India** to join the IBM Quantum Network, providing:

- Preferential access to IBM's latest quantum processors
- Qiskit training and certification pipelines
- Research collaboration on quantum algorithms and error correction
- The foundation for this very course

The Quantum Workforce Gap

Globally there is a severe shortage of quantum-skilled professionals. McKinsey estimates that by 2030, the quantum industry will need 650,000 quantum-trained workers. India has specific advantages:

- Large pool of physics and engineering graduates
- Established IT/software industry for quantum software development
- Growing government investment and startup ecosystem (BosonQ Psi, QNu Labs)

NPTEL CODE Program

This course is part of the NPTEL CODE (Collaborative Online Delivery Enhancement) program — a structured industry-academia collaboration bringing world-class education to learners across India. The program offers:

- Free access to course content
- Optional proctored certification exams
- Industry-recognized certificates from IIT Madras + IBM

Real World Usage

- **QNu Labs (India):** Quantum key distribution for secure communications
- **BosonQ Psi (India):** Quantum simulation for aerospace engineering
- **TCS Quantum Computing Lab:** Exploring quantum algorithms for enterprise applications
- **Infosys Quantum:** Research partnerships with IBM for business applications
- **DRDO:** Quantum cryptography for defense communications

Key Takeaways

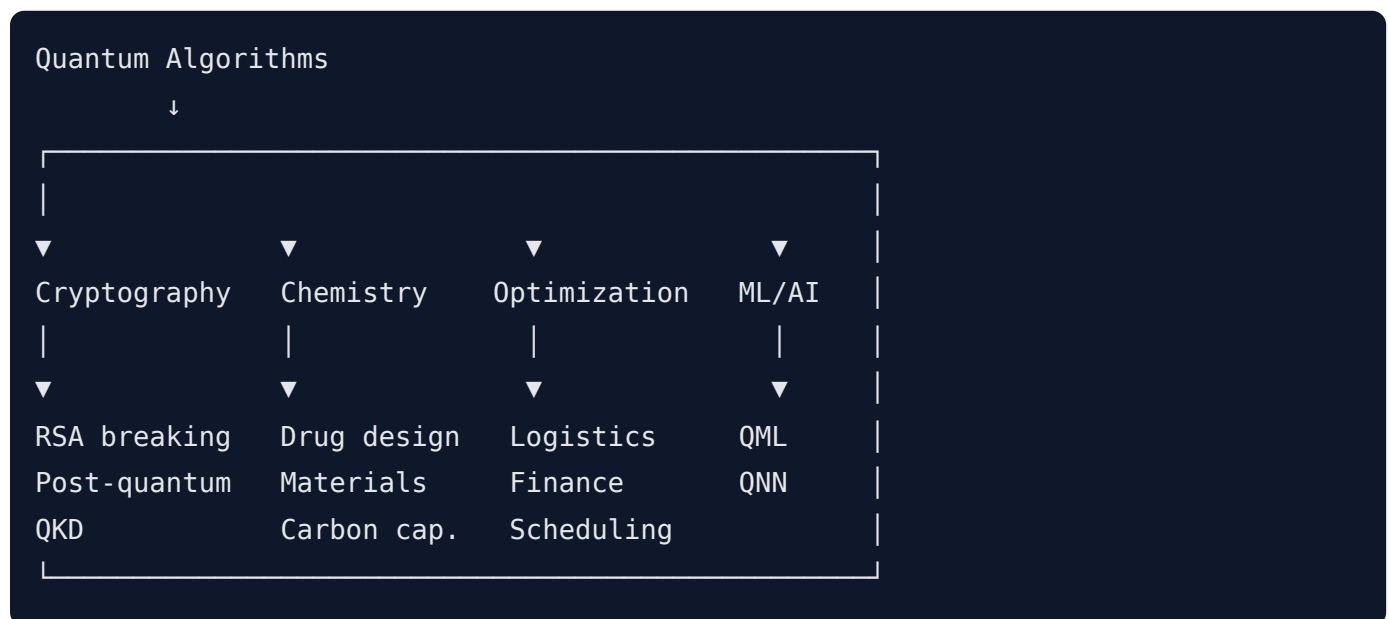
- India is investing heavily in quantum — career opportunities are real and growing
- IIT Madras + IBM collaboration gives this course unique credibility
- The workforce gap is an opportunity for motivated learners
- Quantum skills translate across academia, industry, and government sectors

Lecture 3 — Applications of Quantum Computing

Objective

Survey the real-world problem domains where quantum computing offers genuine advantages. Move beyond hype to concrete, technically grounded examples.

Big Picture



Core Concepts

Application Domain 1: Cryptography

Current threat (Shor's Algorithm):

RSA encryption relies on the fact that factoring a 2048-bit number is computationally intractable classically (takes millions of years). Shor's quantum algorithm factors such numbers in polynomial time. A sufficiently powerful fault-tolerant quantum computer would break RSA, ECC, and Diffie-Hellman encryption.

Timeline: The threat is real but not immediate. Breaking RSA-2048 requires ~4000 logical qubits with error correction, translating to millions of physical qubits. Current devices have ~1000 physical qubits with poor error rates. Estimate: 10–15 years.

"Harvest now, decrypt later": Nation-state adversaries may already be collecting encrypted traffic today to decrypt once quantum computers become powerful enough. This is why governments are standardizing post-quantum cryptography NOW.

Post-quantum cryptography (PQC):

NIST finalized the first post-quantum cryptographic standards in 2024: CRYSTALS-Kyber (key encapsulation) and CRYSTALS-Dilithium (digital signatures), both based on lattice problems that remain hard even for quantum computers.

Quantum Key Distribution (QKD):

Quantum mechanics enables provably secure key exchange. Any eavesdropping on a quantum channel disturbs the quantum states and is detectable. China launched the world's first quantum satellite (Micius) and demonstrated QKD over 1200 km.

Application Domain 2: Quantum Chemistry and Drug Discovery

The problem: Simulating the electronic structure of molecules is exponentially hard classically. The number of classical bits needed to represent a molecule's quantum state grows exponentially with the number of electrons. This is why simulating a caffeine molecule on a classical supercomputer is already challenging.

Quantum advantage: A quantum computer can represent molecular quantum states naturally. The simulation requires resources that scale polynomially with system size.

Impact:

- Design catalysts for nitrogen fixation (cutting 2% of global energy use for fertilizer production)
- Discover new battery materials for electric vehicles
- Simulate protein folding for drug design
- Model carbon capture chemistry

Near-term application (VQE): The Variational Quantum Eigensolver (covered in Week 4) is a NISQ-era hybrid quantum-classical algorithm already being used by companies like Merck, Biogen, and JSR Corporation to explore molecular simulations.

Application Domain 3: Optimization

Many real-world problems are combinatorial: find the best solution among an astronomically large set of possibilities.

Examples:

- **Logistics:** Travelling Salesman Problem — optimal delivery route for 1000 cities
- **Finance:** Portfolio optimization — best mix of 500 assets given risk constraints
- **Scheduling:** Hospital staff scheduling, airline crew scheduling
- **Traffic:** Volkswagen used D-Wave quantum annealing to optimize traffic flow in Lisbon

Quantum approach (QAOA): The Quantum Approximate Optimization Algorithm (QAOA, Week 4) maps optimization problems to quantum circuits and finds approximate good solutions using hybrid quantum-classical loops.

Quantum annealing (D-Wave): A different quantum paradigm that physically mimics the process of cooling a metal to find its lowest energy state, which corresponds to the optimal solution of the encoded problem.

Application Domain 4: Machine Learning

Quantum Machine Learning (QML) explores whether quantum hardware can accelerate ML tasks:

- **Quantum kernel methods:** Quantum computers can compute certain kernel functions exponentially faster
- **Quantum neural networks (QNN):** Parameterized quantum circuits as trainable models
- **Amplitude encoding:** Encode N-dimensional classical data into $\log_2(N)$ qubits

Honest assessment: QML speedups are problem-specific and often require quantum RAM (not yet built) for full theoretical advantage. Active research area with genuine potential but many open questions.

Application Domain 5: Financial Modeling

- Monte Carlo simulations (risk analysis) — quantum speedup with Quantum Amplitude Estimation
- Derivative pricing — BBVA, Goldman Sachs, JPMorgan actively researching
- Fraud detection — pattern recognition in high-dimensional transaction data

Internal Working: Why Quantum Helps Chemistry

Classical simulation of an N-electron system requires tracking 2^N basis states. For 50 electrons, that's $2^{50} \approx 10^{15}$ states — beyond any classical memory. A quantum computer with 50 qubits represents all 2^{50} states simultaneously in superposition, and the quantum algorithm naturally manipulates this exponentially large space using only 50 qubits.

Common Bugs (Conceptual Misconceptions)

Misconception	Reality
"Quantum computers solve all NP problems in polynomial time"	FALSE. Quantum computers are known to solve SOME NP problems faster but are not proven to solve all NP problems efficiently
"QML always provides speedup over classical ML"	FALSE. Speedup depends heavily on the problem structure and data loading costs
"Quantum computers will replace classical computers"	FALSE. They will be specialized coprocessors for specific tasks
"We are 5 years away from breaking RSA"	UNCERTAIN. Requires millions of error-corrected qubits — likely 10–20 years

Interview Questions

Scenario-based:

- You are a cybersecurity manager. What should you do today to prepare for quantum threats?
- A pharma company wants to explore quantum computing for drug design. What algorithm would you recommend and why?
- Explain why quantum annealing is different from gate-model quantum computing.

Key Takeaways

- Quantum speedups are real but problem-specific — always ask "is this the right tool?"
 - Cryptography: near-term threat (QKD + PQC already in deployment)
 - Chemistry: most natural application with clear exponential advantage
 - Optimization and ML: near-term NISQ opportunities with hybrid classical-quantum approaches
 - The honest story is nuanced — avoid both hype and cynicism
-

Lecture 4 — Quantum Computing Basics: Qubits and Linear Algebra

Objective

Build the mathematical foundation of quantum mechanics as it applies to quantum computing: complex vector spaces, the qubit formalism, measurement, and the Dirac notation used throughout the course.

Big Picture

```
Linear Algebra (math)
  ↓
Quantum Mechanics (physics)
  ↓
Qubit Formalism (representation)
  ↓
Quantum Gates (operations)
  ↓
Quantum Circuits (algorithms)
  ↓
Results (measurement)
```

Quantum computing is, at its mathematical core, linear algebra over complex numbers. Understanding this section deeply is the key that unlocks everything else.

Core Concepts

1. Vector Spaces and Hilbert Space

A **vector space** is a set of objects (vectors) with rules for addition and scalar multiplication. In quantum mechanics, the relevant mathematical arena is a **complex Hilbert space** — a vector space over the complex numbers $\mathbb{C}$ with an inner product.

For a single qubit, the Hilbert space is $\mathbb{C}^2$ — the set of all 2-dimensional complex column vectors.

2. Dirac Notation (Bra-Ket Notation)

Paul Dirac introduced an elegant notation that quantum computing universally adopts:

Symbol	Name	Mathematical Object
$ \psi\rangle$	Ket	Column vector (state of a system)
$\langle\psi $	Bra	Row vector (conjugate transpose of ket)
$\langle\phi \psi\rangle$	Bracket / Inner product	Complex scalar
$ \phi\rangle\langle\psi $	Outer product	Matrix (operator)

The two computational basis states of a qubit:

$$\begin{aligned} |0\rangle &= \begin{bmatrix} 1 \\ 0 \end{bmatrix} & |1\rangle &= \begin{bmatrix} 0 \\ 1 \end{bmatrix} \end{aligned}$$

These are orthogonal unit vectors: $\langle 0|0\rangle = 1, \langle 1|1\rangle = 1, \langle 0|1\rangle = 0$.

3. General Qubit State

A general qubit state:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle = \begin{bmatrix} \alpha \\ \beta \end{bmatrix}$$

Where:

- $\alpha, \beta \in \mathbb{C}$ (complex numbers)
- Normalization:** $|\alpha|^2 + |\beta|^2 = 1$
- $|\alpha|^2$ = probability of measuring $|0\rangle$

- $|\beta|^2$ = probability of measuring $|1\rangle$

Example: The state $|+\rangle = (1/\sqrt{2})|0\rangle + (1/\sqrt{2})|1\rangle$ has equal probability (50%) of measuring 0 or 1.

4. The Measurement Postulate

Quantum measurement is **probabilistic** and **destructive**:

Before measurement: $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$

Measure in
computational basis

↓

After measurement: $|0\rangle$ with probability $|\alpha|^2$
 $|1\rangle$ with probability $|\beta|^2$

And the state **COLLAPSES** to the measured value.
 The original superposition is destroyed.

This is called **wavefunction collapse** and is one of the most philosophically debated aspects of quantum mechanics. For the purposes of quantum computing, it is simply a rule: measurement gives a classical output and destroys the quantum state.

5. Phase and Its Importance

Complex amplitudes have a **magnitude** and a **phase** (angle). For example:

$$\begin{array}{ll} \alpha = 1/\sqrt{2} & \beta = i/\sqrt{2} \\ |\alpha|^2 = 1/2 & |\beta|^2 = 1/2 \end{array}$$

Both states would give the same measurement probabilities (50/50), yet they are physically distinct quantum states — their different phases lead to different behaviors under quantum operations (interference). **Phase is not directly measurable** but is crucial for quantum algorithms.

This leads to the concept of **global phase** (physically irrelevant, $e^{i\theta}|\psi\rangle \equiv |\psi\rangle$) vs **relative phase** (physically relevant, affects interference).

6. Multiple Qubits: Tensor Products

Two qubits together live in a 4-dimensional Hilbert space: $\mathbb{C}^2 \otimes \mathbb{C}^2 = \mathbb{C}^4$.

The **tensor product** combines the spaces:

$$\begin{aligned}
|00\rangle &= |0\rangle \otimes |0\rangle = [1, 0, 0, 0]^T \\
|01\rangle &= |0\rangle \otimes |1\rangle = [0, 1, 0, 0]^T \\
|10\rangle &= |1\rangle \otimes |0\rangle = [0, 0, 1, 0]^T \\
|11\rangle &= |1\rangle \otimes |1\rangle = [0, 0, 0, 1]^T
\end{aligned}$$

A general 2-qubit state:

$$|\psi\rangle = \alpha_{00}|00\rangle + \alpha_{01}|01\rangle + \alpha_{10}|10\rangle + \alpha_{11}|11\rangle$$

Where $|\alpha_{00}|^2 + |\alpha_{01}|^2 + |\alpha_{10}|^2 + |\alpha_{11}|^2 = 1$.

For n qubits: 2^n -dimensional Hilbert space. **This exponential growth is the source of quantum power.**

7. Entanglement (Formal Definition)

A two-qubit state is **entangled** if it CANNOT be written as a tensor product of two single-qubit states.

Example — Bell state:

$$|\Phi^+\rangle = (1/\sqrt{2})(|00\rangle + |11\rangle)$$

Attempt to factor: $\alpha|0\rangle \otimes \beta|0\rangle + \dots$ — impossible. This state is entangled.

When you measure the first qubit and get $|0\rangle$, the second qubit is instantly $|0\rangle$. If you get $|1\rangle$, the second is instantly $|1\rangle$. No matter how far apart they are.

8. Unitary Evolution

Between measurements, quantum states evolve according to **unitary operators** (matrices U satisfying $U^\dagger U = I$, where $U^\dagger$ is the conjugate transpose). This ensures:

- Normalization is preserved (probabilities always sum to 1)
- Evolution is reversible (every quantum gate has an inverse)

Internal Working: The Mathematics of Superposition

When we say "50/50 superposition," this is not like flipping a coin where the result exists but is unknown. The mathematics of interference proves the difference:

Classical probability: $P(0) = 0.5$, $P(1) = 0.5$. No interference. Cannot cancel.

Quantum amplitude: $\alpha = 1/\sqrt{2}$ for $|0\rangle$, $\beta = 1/\sqrt{2}$ for $|1\rangle$. Now apply a Hadamard gate:

$$\begin{aligned}
H|+\rangle &= H \cdot (1/\sqrt{2})(|0\rangle + |1\rangle) = (1/\sqrt{2})[H|0\rangle + H|1\rangle] \\
&= (1/\sqrt{2})[(|0\rangle + |1\rangle)/\sqrt{2} + (|0\rangle - |1\rangle)/\sqrt{2}] \\
&= (1/\sqrt{2})[2|0\rangle/\sqrt{2}] \\
&= |0\rangle
\end{aligned}$$

The $|1\rangle$ amplitudes **cancelled** through destructive interference. The $|0\rangle$ amplitudes **reinforced** through constructive interference. This is fundamentally impossible for classical probabilities (probabilities cannot be negative). **Quantum interference is the engine of quantum algorithms.**

Step-by-Step Execution: Measuring a Superposition

- Step 1: Initialize qubit in $|0\rangle$ state
(superconducting qubit set to ground state by cooling)
- Step 2: Apply Hadamard gate
 $|0\rangle \rightarrow (1/\sqrt{2})(|0\rangle + |1\rangle) = |+\rangle$
- Step 3: Quantum state is now in superposition
Physically: qubit's Bloch vector points at equator
- Step 4: Measurement device (readout resonator) probes qubit
- Step 5: Quantum randomness: result is 0 or 1 with equal probability
- Step 6: State collapses to $|0\rangle$ or $|1\rangle$
- Step 7: Classical bit is recorded
- Step 8: Repeating the experiment 1000 times gives ~500 zeros, 500 ones
→ Statistics confirm quantum state was $|+\rangle$

Visual Mental Models

Linear Algebra for Quantum States

The Tensor Product

2 qubits: $\mathbb{C}^2 \otimes \mathbb{C}^2 = \mathbb{C}^4$ (4 basis states)

3 qubits: $\mathbb{C}^2 \otimes \mathbb{C}^2 \otimes \mathbb{C}^2 = \mathbb{C}^8$ (8 basis states)

n qubits: 2^n basis states

A classical register of n bits: n values needed

A quantum register of n qubits: 2^n amplitudes stored simultaneously

n=10: 1024 classical states vs 1024 amplitudes in ONE quantum state

n=50: 50 bits classical vs 10^{15} amplitudes in ONE quantum state

n=300: 300 bits classical vs more amplitudes than atoms in the universe

Common Bugs

Mistake	Explanation
"Measuring gives definite answer that was always there"	Measurement creates the outcome; hidden variable theories ruled out by Bell's theorem
"We can read out all amplitudes"	We can only sample outcomes; extracting all 2^n amplitudes would require exponential measurements
"Global phase matters"	$e^{i\theta}$
Confusing tensor product with Cartesian product	Tensor product creates exponentially larger space; crucial for quantum power

Interview Questions

Basic:

- Write the general form of a single qubit state and explain normalization.
- What is the computational basis?

- What happens when you measure a qubit in superposition?

Intermediate:

- Why is quantum interference impossible to reproduce with classical probabilities?
- Explain the difference between global and relative phase.
- How many basis states does an n-qubit system have and why?

Advanced:

- Define entanglement formally. Give an example of an entangled 2-qubit state and prove it cannot be factored.
- Why must quantum gates be unitary?

Key Takeaways

- Quantum mechanics is linear algebra over complex numbers
- The qubit state $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$ encodes complex amplitudes, not probabilities
- Measurement gives probabilistic outcomes and destroys superposition
- Phase is invisible to measurement but crucial for quantum interference
- n qubits encode 2^n amplitudes — the exponential resource of quantum computing
- Entanglement: correlations that can never exist classically

WEEK 2 — IBM TOOLS AND HARDWARE

Lectures 5–6 — IBM Quantum Composer and IBM Quantum Lab (Qiskit Introduction)

Objective

Learn to use IBM's two cloud quantum tools: the **IBM Quantum Composer** (drag-and-drop circuit builder) and **IBM Quantum Lab** (Jupyter notebook environment with Qiskit). Build, run, and analyze simple quantum circuits on real IBM hardware.

Big Picture

Quantum Algorithm Idea

↓

Qiskit Python Code / IBM Circuit Composer (visual)

↓

QuantumCircuit object in Qiskit

↓

Transpile to native gate set

↓

Submit to IBM Quantum via IBMQ Account

↓

Queue on real device (e.g., ibmq_manila) OR Qasm Simulator

↓

Execute (microwave pulses on superconducting qubits)

↓

Retrieve Results (counts dictionary)

↓

Visualize (histogram, Bloch sphere, state vector)

Core Concepts

IBM Quantum Composer

The Composer is a browser-based drag-and-drop interface:

- Qubit wires run horizontally (left to right = time)
- Gates are dragged onto wires
- Classical registers appear as double lines (for measurement results)
- Click "Run" to execute on a simulator or real device

It is excellent for **learning** and **visualizing** but limits are felt quickly for complex circuits. Most professional work uses Qiskit directly.

Qiskit Overview

Qiskit is IBM's open-source quantum computing SDK written in Python. It has four main elements (though this has been restructured in recent versions):

Package	Purpose
qiskit-terra	Core circuit building, transpilation
qiskit-aer	Simulation (noise models, state vector)
qiskit-ibmq-provider	Cloud access to IBM hardware

Modern Qiskit (1.x and beyond) has unified these into `qiskit` + `qiskit-ibm-runtime`.

Building Your First Circuit

```
from qiskit import QuantumCircuit, QuantumRegister, ClassicalRegister

# Create a circuit with 2 qubits and 2 classical bits
qr = QuantumRegister(2, 'q')      # 2 quantum registers
cr = ClassicalRegister(2, 'c')    # 2 classical registers
qc = QuantumCircuit(qr, cr)      # compose into circuit

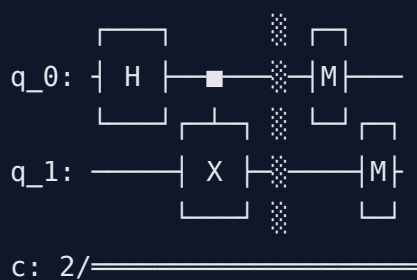
# Apply Hadamard gate to qubit 0 → creates superposition
qc.h(qr[0])

# Apply CNOT gate: qubit 0 (control), qubit 1 (target) → creates entanglement
qc.cx(qr[0], qr[1])

# Measure both qubits
qc.measure(qr, cr)

# Draw the circuit
print(qc.draw())
```

Circuit diagram output:



Running on Simulator

```

from qiskit_aer import AerSimulator
from qiskit import transpile

# Use the Aer statevector simulator
simulator = AerSimulator()

# Transpile the circuit for the simulator
compiled = transpile(qc, simulator)

# Run 1024 shots
job = simulator.run(compiled, shots=1024)
result = job.result()
counts = result.get_counts()
print(counts) # e.g., {'00': 512, '11': 512}

```

The result `{'00': 512, '11': 512}` confirms a Bell state — you only ever measure `00` or `11`, never `01` or `10`. This is the signature of entanglement.

Running on Real IBM Hardware

```

from qiskit_ibm_runtime import QiskitRuntimeService

# Save account (one-time)
QiskitRuntimeService.save_account(channel='ibm_quantum', token='YOUR_API_TOKEN')

# Load account and find a real backend
service = QiskitRuntimeService()
backend = service.least_busy(operational=True, simulator=False)
print(backend.name)

# Run on real device
from qiskit_ibm_runtime import SamplerV2 as Sampler
sampler = Sampler(backend)
job = sampler.run([transpile(qc, backend)])
result = job.result()

```

Detailed Explanation: Key Quantum Gates

Single-Qubit Gates (2×2 Unitary Matrices)

Pauli-X gate (NOT gate):


```
X = [0  1]   X|0> = |1>   X|1> = |0>
     [1  0]
```

```
qc.x(0)  # Flip qubit 0
```

Pauli-Z gate (Phase flip):

```
Z = [1  0]   Z|0> = |0>   Z|1> = -|1>
     [0 -1]
```

```
qc.z(0)  # Flip phase of |1> component
```

Hadamard gate (H gate) — creates superposition:

```
H = (1/√2)[1  1]   H|0> = |+> = (|0>+|1>)/√2
          [1 -1]   H|1> = |-> = (|0>-|1>)/√2
```

```
qc.h(0)  # Put qubit 0 in superposition
```

Rotation gates (general single-qubit rotations):

```
qc.rx(theta, 0)  # Rotate around X-axis by theta radians
qc.ry(theta, 0)  # Rotate around Y-axis by theta radians
qc.rz(theta, 0)  # Rotate around Z-axis by theta radians
```

The RZ gate is native on IBM hardware; X and H are decomposed into RZ and SX gates during transpilation.

Two-Qubit Gates

CNOT (Controlled-NOT, CX) gate — entangles qubits:

```
CX|00> = |00>
CX|01> = |01>
CX|10> = |11>  ← Target flips when control is |1>
CX|11> = |10>

qc.cx(control=0, target=1)
```

Matrix representation (4×4 , in basis $|00\rangle, |01\rangle, |10\rangle, |11\rangle$):

```
CX = [1 0 0 0]
      [0 1 0 0]
      [0 0 0 1]
      [0 0 1 0]
```

CNOT is the standard entangling gate. Combined with single-qubit gates, it forms a **universal gate set** — any quantum computation can be expressed using only H, CNOT, and T gates.

Internal Working: How IBM Transpilation Works

When you write `qc.h(0)`, Qiskit represents this abstractly. Before running on real hardware, the **transpiler** must:

1. **Decompose** high-level gates into native gates (IBM native set: {ID, RZ, SX, X, CX})
2. **Map** logical qubits to physical qubits on the specific chip (qubit connectivity is limited)
3. **Route** gates that require SWAP operations if two connected qubits are required but not physically adjacent
4. **Optimize** to minimize circuit depth (reduce decoherence errors)

```
User writes:  H(q0) - CNOT(q0→q1) - Measure
```

```
Transpiler:
```

1. $H(q_0) \rightarrow RZ(\pi/2, q_0) \cdot SX(q_0) \cdot RZ(\pi/2, q_0)$
2. $CNOT(q_0 \rightarrow q_1) \rightarrow$ native CX if q_0, q_1 are connected;
otherwise SWAP + CX + SWAP
3. Optimize: combine adjacent RZ gates

```
Final circuit: Lower-level, hardware-specific
```

Step-by-Step Execution: Bell State Creation

Step 1: Initialize: both qubits in $|0\rangle|0\rangle = |00\rangle$
 State vector: $[1, 0, 0, 0]$

Step 2: Apply H to qubit 0:
 $|\psi\rangle = (1/\sqrt{2})(|0\rangle+|1\rangle)|0\rangle = (1/\sqrt{2})(|00\rangle+|10\rangle)$
 State vector: $[1/\sqrt{2}, 0, 1/\sqrt{2}, 0]$

Step 3: Apply CNOT (control q0, target q1):
 $|00\rangle \rightarrow |00\rangle$ (control=0, target unchanged)
 $|10\rangle \rightarrow |11\rangle$ (control=1, target flips)
 $|\psi\rangle = (1/\sqrt{2})(|00\rangle+|11\rangle) \leftarrow \text{Bell state } |\Phi^+\rangle$
 State vector: $[1/\sqrt{2}, 0, 0, 1/\sqrt{2}]$

Step 4: Measure:
 $P(00) = |1/\sqrt{2}|^2 = 1/2$
 $P(11) = |1/\sqrt{2}|^2 = 1/2$
 $P(01) = 0, \quad P(10) = 0$

Result: Always get 00 or 11, never 01 or 10.

Visual Mental Models

Quantum Circuit Notation

Time flows LEFT $\rightarrow$ RIGHT

```

q_0: —[H]—■——[M]—
          |
q_1: ——[CX]——[M]—
  
```

[H] = Hadamard gate

■ = Control qubit of CNOT

[CX] = Target qubit of CNOT (X applied conditionally)

[M] = Measurement (collapses to classical bit)

Bell State Measurement Statistics

Run 1000 shots on Bell state $|\Phi^+\rangle$:

Outcome	Count	Probability
00	~500	50%
01	0	0%
10	0	0%
11	~500	50%

(Real hardware: slight deviations due to noise)

Code Breakdown: Bell State Circuit

```

from qiskit import QuantumCircuit          # Line 1
# Imports QuantumCircuit class from Qiskit.
# Without this import, Python has no knowledge of quantum circuits.

qc = QuantumCircuit(2, 2)                  # Line 2
# Creates a circuit with 2 quantum bits (indices 0, 1)
# and 2 classical bits (indices 0, 1).
# If removed: no circuit object to add gates to.
# Memory: QuantumCircuit object stored in Python heap.

qc.h(0)                                    # Line 3
# Appends a Hadamard gate on qubit 0 to the circuit.
# Does NOT execute anything yet – just adds to circuit description.
# Effect:  $|0\rangle \rightarrow (|0\rangle + |1\rangle)/\sqrt{2}$ 
# Time complexity:  $O(1)$  to add;  $O(1)$  to execute (single  $2 \times 2$  matrix multiply).

qc.cx(0, 1)                                # Line 4
# Appends CNOT gate: control=qubit0, target=qubit1.
# Creates entanglement between the two qubits.
# The CNOT is the fundamental two-qubit entangling gate.
# Without this: state is a product state, not entangled.

qc.measure([0, 1], [0, 1])                # Line 5
# Adds measurement operations:
# qubit 0  $\rightarrow$  classical bit 0
# qubit 1  $\rightarrow$  classical bit 1
# Measurement collapses quantum state to classical bits.
# Without this: circuit has no output; can't read results.

```

Common Bugs

Bug	Cause	Fix
<code>IBMQAccountCredentialsNotFound</code>	API token not saved	Run <code>QiskitRuntimeService.save_account(...)</code>
Circuit depth too large	Too many gates; decoherence	Optimize circuit; use fewer gates
<code>{'00': 1024}</code> even for Bell state	Forgot H gate	Check circuit with <code>qc.draw()</code>

Results look noisy on real device	Actual quantum noise	Increase shots; use error mitigation
<code>TranspilerError</code>	Gate not in native set	Let Qiskit transpile automatically

Interview Questions

Basic:

- What does the Hadamard gate do to $|0\rangle$? What is the resulting state?
- What is a CNOT gate and when does it flip the target qubit?
- What is the difference between a quantum register and a classical register in Qiskit?

Intermediate:

- Explain why the Bell state $|\Phi^+\rangle = (|00\rangle + |11\rangle)/\sqrt{2}$ never gives outcomes 01 or 10.
- What is transpilation and why is it necessary?
- How many shots should you run and why?

Advanced:

- Why are H + CNOT + T gates universal?
- Explain the circuit connectivity constraint of IBM superconducting processors. How does this affect circuit design?

Key Takeaways

- IBM Quantum Composer: drag-and-drop for learning; Qiskit: Python for real work
- `QuantumCircuit` is the central Qiskit object — gates are appended, then transpiled and run
- H gate creates superposition; CNOT creates entanglement
- Bell states are the simplest entangled states and the signature of quantum correlations
- Real hardware has noise; simulation is perfect but not physical

Lectures 7–9 — Quantum Gates Deep Dive, Bell States, No-Cloning Theorem, and Quantum Teleportation

Objective

Master the full single and two-qubit gate vocabulary, understand the four Bell states, prove the no-cloning theorem, and understand quantum teleportation as the premier demonstration of entanglement as a resource.

Core Concepts

The Four Bell States

Bell states are the four maximally entangled 2-qubit states:

$$|\Phi^+\rangle = (1/\sqrt{2})(|00\rangle + |11\rangle) \quad \leftarrow \text{Symmetric, same parity}$$
$$|\Phi^-\rangle = (1/\sqrt{2})(|00\rangle - |11\rangle) \quad \leftarrow \text{Antisymmetric phase, same parity}$$
$$|\Psi^+\rangle = (1/\sqrt{2})(|01\rangle + |10\rangle) \quad \leftarrow \text{Symmetric, opposite parity}$$
$$|\Psi^-\rangle = (1/\sqrt{2})(|01\rangle - |10\rangle) \quad \leftarrow \text{Antisymmetric phase, opposite parity}$$

They form an **orthonormal basis** for the 4-dimensional 2-qubit Hilbert space — any 2-qubit state can be expressed as a superposition of Bell states.

Creating Bell states from $|00\rangle$:

Start	Operations	Bell State
	$ 00\rangle$	H(q0), CNOT
	$ 01\rangle$	H(q0), CNOT
	$ 10\rangle$	H(q0), CNOT
	$ 11\rangle$	H(q0), CNOT

The No-Cloning Theorem

Statement: It is impossible to create an identical copy of an arbitrary unknown quantum state.

Proof by contradiction:

Suppose a unitary operator U exists that clones states: $U|\psi\rangle|0\rangle = |\psi\rangle|\psi\rangle$ for all $|\psi\rangle$.

Apply to $|0\rangle$ and $|1\rangle$:

$$U|0\rangle|0\rangle = |0\rangle|0\rangle \quad \dots \text{ (i)}$$
$$U|1\rangle|0\rangle = |1\rangle|1\rangle \quad \dots \text{ (ii)}$$

By linearity, for $|+\rangle = (|0\rangle+|1\rangle)/\sqrt{2}$:

$$\begin{aligned}
 U|+\rangle|0\rangle &= (1/\sqrt{2})(U|0\rangle|0\rangle + U|1\rangle|0\rangle) \\
 &= (1/\sqrt{2})(|00\rangle + |11\rangle) = |\Phi^+\rangle \text{ (Bell state)}
 \end{aligned}$$

But cloning would require:

$$U|+\rangle|0\rangle = |+\rangle|+\rangle = (1/2)(|00\rangle + |01\rangle + |10\rangle + |11\rangle)$$

These are different states → **contradiction**. No such U exists. QED.

Implications:

- Cannot perfectly copy quantum information (unlike classical photocopying)
- Secures quantum cryptography (any eavesdropping is detectable — eavesdropper can't clone the quantum key)
- Necessitates quantum teleportation (which moves, not copies, quantum state)

Quantum Teleportation

The problem: Alice wants to send an unknown qubit state $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$ to Bob. They cannot copy it (no-cloning). They are far apart.

Solution (using entanglement + classical communication):

Pre-condition: Alice and Bob share a Bell pair $|\Phi^+\rangle$ (created in advance and distributed).

Initial state:

Alice has: $|\psi\rangle$ (qubit to teleport) + her half of $|\Phi^+\rangle$

Bob has: his half of $|\Phi^+\rangle$

Full state (3 qubits):

$$|\psi\rangle_A \otimes |\Phi^+\rangle_{AB} = (\alpha|0\rangle + \beta|1\rangle)_A \otimes (1/\sqrt{2})(|00\rangle + |11\rangle)_{BC}$$

Protocol:

Step 1: Alice applies CNOT ($|\psi\rangle$ as control, her Bell qubit as target)

Step 2: Alice applies H to $|\psi\rangle$

Step 3: Alice measures her 2 qubits $\rightarrow$ gets classical bits $(m_1, m_2) \in \{00, 01, 10, 11\}$

$\rightarrow$ This collapses Bob's qubit to one of 4 states

Step 4: Alice sends 2 classical bits (m_1, m_2) to Bob (classical channel, can be public)

Step 5: Bob applies corrections:

if $m_2=1$: apply X gate

if $m_1=1$: apply Z gate

$\rightarrow$ Bob's qubit is now exactly $\alpha|0\rangle + \beta|1\rangle = |\psi\rangle$

Remarkable facts:

- Alice's original qubit is destroyed (she measured it — no-cloning respected)
- Bob's qubit becomes $|\psi\rangle$ WITHOUT Bob knowing what α and β are
- The quantum state is "teleported" — the amplitudes travel through entanglement
- 2 classical bits were transmitted, but this does NOT allow faster-than-light communication (Bob doesn't know which correction to apply until classical bits arrive)

Internal Working: Why Teleportation Works

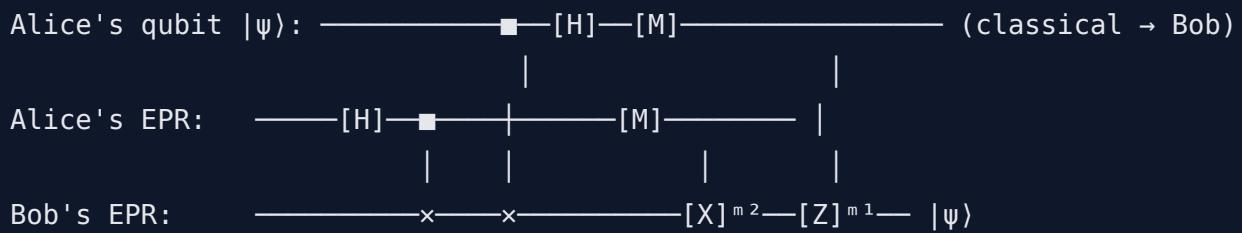
The mathematical insight is a **Bell basis measurement**. The 3-qubit state can be rewritten as:

$$\begin{aligned} |\psi\rangle_A |\Phi^+\rangle_{BC} &= (1/2) [\\ &|\Phi^+\rangle_{AB} \otimes (\alpha|0\rangle + \beta|1\rangle)_C && \leftarrow \text{Alice measures } \Phi^+, \text{ Bob has } |\psi\rangle \text{ already} \\ &|\Phi^-\rangle_{AB} \otimes (\alpha|0\rangle - \beta|1\rangle)_C && \leftarrow \text{Alice measures } \Phi^-, \text{ Bob applies Z} \\ &|\Psi^+\rangle_{AB} \otimes (\alpha|1\rangle + \beta|0\rangle)_C && \leftarrow \text{Alice measures } \Psi^+, \text{ Bob applies X} \\ &|\Psi^-\rangle_{AB} \otimes (\alpha|1\rangle - \beta|0\rangle)_C && \leftarrow \text{Alice measures } \Psi^-, \text{ Bob applies XZ} \\ &] \end{aligned}$$

Alice's Bell measurement reveals which of the 4 cases occurred. The 2 classical bits she sends tell Bob exactly which correction to apply. The quantum amplitudes α and β were "pre-distributed" through the shared entanglement — that's how the information got to Bob's side without traveling through space.

Visual Mental Models

Teleportation Circuit



No-Cloning Visualized

Classical copier:

$|\text{document}\rangle \rightarrow [\text{COPY}] \rightarrow |\text{document}\rangle + |\text{document}\rangle \quad \checkmark$ (allowed)

Quantum "cloner" (impossible):

$|\psi\rangle \rightarrow [\text{COPY?}] \rightarrow |\psi\rangle \otimes |\psi\rangle \quad \times$ (violates linearity of quantum mechanics)

Why? Superposition states would need to be cloned, but linearity prevents copying unknown superpositions.

Common Bugs

Mistake	Correction
"Teleportation violates no-cloning"	No — the original state is destroyed during measurement
"Teleportation allows faster-than-light communication"	No — Bob needs the classical bits before he can apply corrections
"No-cloning means quantum computers can't copy data"	We can copy known classical information encoded in qubits; only unknown quantum states can't be cloned

Interview Questions

Basic:

- State the no-cloning theorem and give an intuitive explanation.
- What are the four Bell states?
- What resources are needed for quantum teleportation?

Intermediate:

- Prove the no-cloning theorem using linearity.
- Why does quantum teleportation not enable faster-than-light communication?
- How many classical bits are communicated in teleportation? Why is this necessary?

Advanced:

- What is quantum dense coding (superdense coding)? How does it use the same Bell pair resource in the reverse direction?
- How is teleportation used in quantum error correction and quantum networks?

Key Takeaways

- Bell states are the four maximally entangled 2-qubit states; basis for quantum information protocols
- No-cloning theorem: unknown quantum states cannot be copied — fundamental, not a technological limitation
- Quantum teleportation moves (not copies) a quantum state using entanglement + 2 classical bits
- Teleportation is the prototype for quantum networks and distributed quantum computing

Lecture 10 — Programming Using IBM Quantum Experience and Circuit Composer

Objective

Develop hands-on proficiency with the IBM Quantum web tools. Learn to use the Circuit Composer for visual circuit building and the IBM Quantum Lab for Jupyter-based Qiskit programming. Understand how to navigate the IBM Quantum platform to access real devices.

Core Concepts

IBM Quantum Platform Architecture

```
quantum.ibm.com
├─ Dashboard (see available backends, queues)
├─ Circuit Composer (visual drag-and-drop)
├─ Quantum Lab (Jupyter notebooks + Qiskit)
├─ Quantum Docs (tutorials, API reference)
└─ IBM Quantum Network (academic/industry partners)
```

IBM Quantum Backends

Backend Type	Purpose	Noise	Availability
qasm_simulator	Noiseless simulation	None	Local/Cloud
ibmq_qasm_simulator	Cloud noiseless sim	None	Free cloud

ibmq_manila (5 qubits)	Small real device	Yes	Free tier
ibmq_kyoto (127 qubits)	Eagle processor	Yes	Premium
ibmq_brisbane (127 qubits)	Eagle processor	Yes	Premium

Key Qiskit Concepts: Jobs and Results

```
# Running a job is asynchronous – you submit and wait
job = backend.run(circuit, shots=1024)

# Check job status
print(job.status()) # QUEUED, RUNNING, DONE, ERROR

# Get results when complete
result = job.result()
counts = result.get_counts() # dict: {'00': 513, '11': 511}

# Visualize histogram
from qiskit.visualization import plot_histogram
plot_histogram(counts)
```

Understanding Shots

In quantum computing, you can't get the probability distribution from a single measurement — each measurement gives one classical outcome. Running **many shots** (typically 1024–8192) samples the probability distribution.

```
Theoretical Bell state probabilities: P(00) = 0.5, P(11) = 0.5

With 1024 shots: {'00': ~512, '11': ~512}
With 100 shots:  {'00': ~50, '11': ~50} (more statistical noise)
With 1 shot:      {'00': 1} or {'11': 1} (useless – can't estimate probability)

Standard error scales as 1/√(shots)
For 1% accuracy: need ~10,000 shots
```

Quantum State Visualization

```

from qiskit import QuantumCircuit
from qiskit.quantum_info import Statevector
from qiskit.visualization import plot_bloch_multivector, plot_state_city

# Create circuit WITHOUT measurement (so we can see state vector)
qc = QuantumCircuit(1)
qc.h(0)

# Get state vector (only possible in simulation)
sv = Statevector.from_instruction(qc)
print(sv)    # [0.707+0j, 0.707+0j]

# Visualize on Bloch sphere
plot_bloch_multivector(sv)

# Bar chart of amplitudes (city plot)
plot_state_city(sv)

```

Internal Working: The IBM Job Queue

When you submit a job to a real IBM quantum backend, it enters a queue:

```

Your Job → IBM Quantum Cloud → Backend Queue
      ↓
    Other users' jobs ahead of you
      ↓
    Hardware scheduler
      ↓
    Physical qubits execute your circuit
    (microwave pulses, ~microseconds)
      ↓
    Measurement, digitization
      ↓
    Results stored in IBM cloud
      ↓
    You retrieve via API

```

Queue times vary from minutes to hours depending on backend demand. IBM shows estimated wait time.

Key Takeaways

- IBM Quantum Platform provides both visual (Composer) and programmatic (Qiskit/Lab) interfaces
- Always start with simulation before running on real hardware (save queue time)
- `shots` controls statistical accuracy — more shots, better statistics, but longer runtime
- State visualization (Bloch sphere, city plots) helps understand what your circuit does
- Real hardware adds noise — compare simulator and hardware results to understand noise impact

WEEK 3 — QUANTUM ALGORITHMS

Lectures 11–13 — Quantum Algorithms I: Oracles and the Deutsch-Jozsa Algorithm

Objective

Introduce the concept of quantum oracles and implement the Deutsch-Jozsa algorithm — the first quantum algorithm to demonstrate a clear speedup over any classical algorithm. This lecture establishes the pattern all oracle-based quantum algorithms follow.

Big Picture

```
Problem defined as a function  $f: \{0,1\}^n \rightarrow \{0,1\}$ 
    ↓
Encode  $f$  as a quantum oracle (black box  $U_f$ )
    ↓
Quantum algorithm queries oracle cleverly
(all inputs simultaneously via superposition)
    ↓
Quantum interference reveals global property
(without examining all inputs)
    ↓
Exponentially fewer oracle queries than classical
```

Core Concepts

The Oracle Model of Computation

In theoretical computer science, an **oracle** is a black-box function that computes $f(x)$ in one step. We count the number of queries to the oracle (query complexity), not arithmetic operations. This lets us compare quantum and classical algorithms fairly.

A **quantum oracle** is a unitary operator U_f defined as:

$$U_f |x\rangle |y\rangle = |x\rangle |y \oplus f(x)\rangle$$

Where:

- x is the n -bit input (encoded in n qubits)
- y is a 1-bit "target" register
- $\oplus$ is XOR (addition mod 2)
- $f(x) \in \{0, 1\}$ is the function value

Why XOR? This form is reversible (required for unitarity). Since $U_f|x\rangle|f(x)\rangle = |x\rangle|0\rangle$, applying U_f twice returns to the original state.

Phase Kickback — The Key Trick

The most powerful use of oracles involves setting $y = |-\rangle = (|0\rangle - |1\rangle)/\sqrt{2}$:

$$\begin{aligned} U_f |x\rangle |-\rangle &= U_f |x\rangle \cdot (1/\sqrt{2})(|0\rangle - |1\rangle) \\ &= (1/\sqrt{2})(|x\rangle |f(x) \oplus 0\rangle - |x\rangle |f(x) \oplus 1\rangle) \\ &= |x\rangle \cdot (1/\sqrt{2})(|f(x)\rangle - |1 \oplus f(x)\rangle) \end{aligned}$$

Case $f(x) = 0$:

$$= |x\rangle \cdot (1/\sqrt{2})(|0\rangle - |1\rangle) = |x\rangle |-\rangle$$

Case $f(x) = 1$:

$$= |x\rangle \cdot (1/\sqrt{2})(|1\rangle - |0\rangle) = -|x\rangle |-\rangle$$

Combined: $U_f|x\rangle|-\rangle = (-1)^{f(x)} \cdot |x\rangle|-\rangle$

The function value $f(x)$ has been encoded into the **phase** of $|x\rangle$. The target qubit is unchanged. This is called **phase kickback** and it is used in virtually every oracle-based quantum algorithm.

The Deutsch-Jozsa Problem

Problem: You are given a function $f: \{0,1\}^n \rightarrow \{0,1\}$. You are promised that f is either:

- **Constant:** $f(x) = 0$ for all x , OR $f(x) = 1$ for all x
- **Balanced:** $f(x) = 0$ for exactly half the inputs, $f(x) = 1$ for the other half

Question: Which is it — constant or balanced?

Classical deterministic: Must query f at least $2^{(n-1)} + 1$ times in the worst case (must check more than half the inputs to be certain).

Classical randomistic: Can be wrong with some probability using fewer queries.

Quantum (Deutsch-Jozsa): ONE oracle query — exponential speedup.

Deutsch-Jozsa Algorithm

Circuit (n input qubits + 1 ancilla):

```
Initialize:
  Input qubits:  $|0\rangle^{\otimes n}$ 
  Ancilla:  $|1\rangle$ 

Apply H to all:
  Input:  $|+\rangle^{\otimes n} = (1/\sqrt{2^n}) \sum_x |x\rangle$ 
  Ancilla:  $|-\rangle$ 

Apply oracle  $U_f$ :
   $(1/\sqrt{2^n}) \sum_x (-1)^{f(x)} |x\rangle |-\rangle$ 

Apply H to input qubits again:
   $H^{\otimes n}$  acts on input register

Measure input qubits:
  All zeros  $\rightarrow f$  is CONSTANT
  Any non-zero  $\rightarrow f$  is BALANCED
```

Mathematical derivation:

After oracle:

$$|\psi\rangle = (1/\sqrt{2^n}) \sum_x (-1)^{f(x)} |x\rangle |-\rangle$$

After second Hadamard on n qubits:

$$H^{\otimes n} |x\rangle = (1/\sqrt{2^n}) \sum_y (-1)^{(x \cdot y)} |y\rangle$$

So:

$$|\psi_{\text{final}}\rangle = (1/2^n) \sum_y [\sum_x (-1)^{(f(x)+x \cdot y)}] |y\rangle$$

For $y = 0$: Amplitude = $(1/2^n) \sum_x (-1)^{f(x)}$

- If constant ($f=0$): amplitude = 1 $\rightarrow$ measure $|0\dots 0\rangle$ with certainty
- If constant ($f=1$): amplitude = -1 $\rightarrow$ still measure $|0\dots 0\rangle$ with certainty (global phase)
- If balanced: amplitudes cancel to 0 $\rightarrow |0\dots 0\rangle$ never measured

For $y \neq 0$: Non-zero amplitude only if balanced.

Result: Measuring all zeros means CONSTANT; any non-zero means BALANCED. **One query. Exact answer.**

Code Breakdown: Deutsch-Jozsa in Qiskit

```

from qiskit import QuantumCircuit
import numpy as np

def deutsch_jozsa(n, oracle_type='constant_0'):
    """
    n: number of input bits
    oracle_type: 'constant_0', 'constant_1', or 'balanced'
    """

    # n input qubits + 1 ancilla qubit
    # n classical bits for measurement
    qc = QuantumCircuit(n + 1, n)

    # Step 1: Set ancilla to |1>
    qc.x(n)          # X gate on last qubit: |0> → |1>

    # Step 2: Apply H to all qubits
    for qubit in range(n + 1):
        qc.h(qubit)  # Creates  $|+\rangle^n \otimes |-\rangle$ 

    qc.barrier()      # Visual separator, no physical effect

    # Step 3: Apply oracle
    if oracle_type == 'constant_0':
        pass          #  $f(x)=0$  for all  $x \rightarrow$  identity oracle  $\rightarrow$  do nothing

    elif oracle_type == 'constant_1':
        qc.x(n)        #  $f(x)=1$  for all  $x \rightarrow$  flip ancilla always
        # Phase kickback: adds global phase  $(-1)$  to all  $|x\rangle$ 

    elif oracle_type == 'balanced':
        # Example balanced oracle:  $f(x) = x_0$  (first bit)
        qc.cx(0, n)     # CNOT: control=input[0], target=ancilla
        #  $f(x)=x_0$  means half of inputs give 0, half give 1  $\rightarrow$  balanced

    qc.barrier()

    # Step 4: Apply H to input qubits again (not ancilla)
    for qubit in range(n):
        qc.h(qubit)

    # Step 5: Measure input qubits only

```

```

qc.measure(range(n), range(n))

return qc

# Test with n=3 input qubits, balanced oracle
circuit = deutsch_jozsa(3, oracle_type='balanced')
print(circuit.draw())

```

Running and interpreting:

```

from qiskit_aer import AerSimulator
from qiskit import transpile

simulator = AerSimulator()
compiled = transpile(circuit, simulator)
job = simulator.run(compiled, shots=1024)
counts = job.result().get_counts()
print(counts)
# Expected: {'001': 1024} or similar non-zero string → BALANCED
# If '000': 1024 → CONSTANT

```

Internal Working: Quantum Parallelism and Interference

The Deutsch-Jozsa algorithm exploits two quantum phenomena:

1. Quantum parallelism:

The Hadamard gate creates a superposition of all 2^n inputs simultaneously. When the oracle U_f is applied, it evaluates $f(x)$ for ALL 2^n inputs in ONE operation.

BUT — we can't directly read all 2^n results. Measurement gives only one outcome.

2. Quantum interference:

The second set of Hadamard gates creates interference among the 2^n output amplitudes:

- For CONSTANT f : all amplitudes interfere constructively at $|0\dots0\rangle$ and destructively elsewhere
- For BALANCED f : amplitudes at $|0\dots0\rangle$ cancel completely (destructive interference)

The measurement then reveals this global property.

Why classical algorithms can't do this:

A classical computer queries $f(x)$ one input at a time. With $n=100$ input bits, there are 2^{100} inputs — impossible to check all. Quantum mechanics doesn't "check all" either in a classical sense; instead, it uses the mathematics of interference over all possibilities simultaneously.

Step-by-Step Execution

Initial state: $|0\rangle^n |1\rangle$

Step 1: X on ancilla: $|0\rangle^n |1\rangle \rightarrow$ no change (already $|1\rangle$)

Step 2: $H^{\otimes(n+1)}$:

Input: $(1/\sqrt{2^n}) \sum_{x=0}^{2^n-1} |x\rangle$

Ancilla: $|-\rangle = (|0\rangle - |1\rangle)/\sqrt{2}$

Step 3: Oracle (phase kickback):

$(1/\sqrt{2^n}) \sum_x (-1)^{f(x)} |x\rangle |-\rangle$

Step 4: $H^{\otimes n}$ on input:

$(1/2^n) \sum_y [\sum_x (-1)^{(f(x) \oplus x \cdot y)}] |y\rangle |-\rangle$

Step 5: Measure input:

Outcome $|0\dots 0\rangle \rightarrow$ CONSTANT with prob 1

Outcome $\neq |0\dots 0\rangle \rightarrow$ BALANCED with prob 1

Real World Usage

- **IBM Research:** Deutsch-Jozsa demonstrated quantum speedup before any other algorithm; now used as a benchmarking circuit
- **Quantum compilation research:** Phase kickback technique used in Grover's, Shor's, HHL
- **Teaching tool:** Conceptually cleanest demonstration of quantum advantage over classical computation

Interview Questions

Basic:

- What is the Deutsch-Jozsa problem? What is promised about the function?
- How many oracle queries does the classical algorithm need vs. the quantum algorithm?

Intermediate:

- Explain phase kickback. How does the oracle affect the phase rather than the amplitude?
- Why does measuring all-zeros indicate a constant function?

Advanced:

- The Deutsch-Jozsa problem is a "promise problem" — why does this matter? Could a real-world function be neither constant nor balanced?
- Extend your analysis: can this speedup be proven to be impossible classically?

Key Takeaways

- A quantum oracle encodes $f(x)$ as a unitary: $U_f|x\rangle|y\rangle = |x\rangle|y \oplus f(x)\rangle$
- Phase kickback: setting ancilla to $|-\rangle$ encodes $f(x)$ into the phase of $|x\rangle$
- Deutsch-Jozsa uses 1 oracle query vs. $2^{(n-1)+1}$ classically — provably exponential quantum speedup
- The algorithm works through quantum parallelism (evaluating all inputs at once) and interference (distinguishing constant from balanced by amplitude cancellation)

Lectures 14–16 — Quantum Algorithms II: Grover's Search Algorithm with Hands-On Qiskit

Objective

Understand and implement Grover's algorithm — the canonical example of quantum speedup for unstructured search — achieving $O(\sqrt{N})$ query complexity versus $O(N)$ classically. Implement on Qiskit and observe the speedup experimentally.

Big Picture

Unstructured search: N items, 1 marked

↓

Classical: Check items one by one $\rightarrow O(N)$ queries

↓

Grover's quantum algorithm:

1. Initialize uniform superposition (all items equally likely)
2. Oracle: mark the target item with a phase flip
3. Diffusion: amplify the marked item's amplitude
4. Repeat $\sim \pi/4 \cdot \sqrt{N}$ times
5. Measure $\rightarrow$ find marked item with high probability

↓

$O(\sqrt{N})$ queries — quadratic speedup

Core Concepts

The Search Problem

Given: A search space of $N = 2^n$ items, one of which is "marked" (satisfies some condition). The condition is checked by an oracle O_n :

$$\begin{aligned} O_n |x\rangle &= -|x\rangle && \text{if } x \text{ is the marked item (target)} \\ O_n |x\rangle &= |x\rangle && \text{otherwise} \end{aligned}$$

This is a **phase oracle** — it marks the target by flipping its amplitude's sign.

Classical search: In the worst case, you check all N items. On average, you check $N/2$. This is $O(N)$.

Grover's algorithm: $O(\sqrt{N})$ queries. For $N = 10^6$, classical needs $\sim 10^6$ queries, Grover needs $\sim 10^3$. For $N = 10^{12}$, Grover needs $\sim 10^6$.

Grover's Algorithm: The Three Components

Component 1: Initialization

Start all n qubits in the uniform superposition:

$$|s\rangle = H^{\otimes n} |0\dots 0\rangle = (1/\sqrt{N}) \sum_x |x\rangle$$

Every item has equal probability amplitude $1/\sqrt{N} \rightarrow$ equal probability $1/N$ of being measured.

Component 2: The Oracle (Phase Flip)

$$O_n |x\rangle = (-1)^{f(x)} |x\rangle$$

Where $f(x) = 1$ if x is the marked item, 0 otherwise.

After one oracle application, the marked item's amplitude becomes $-1/\sqrt{N}$ while all others remain $1/\sqrt{N}$. This changes the phase but NOT the probability ($|-1/\sqrt{N}|^2 = 1/N$ still).

Component 3: The Diffusion Operator (Amplitude Amplification)

The diffusion operator reflects all amplitudes about their average:

$$D = 2|s\rangle\langle s| - I$$

Where $|s\rangle = (1/\sqrt{N}) \sum_x |x\rangle$ is the uniform superposition.

Geometric interpretation:

- The oracle reflects the state vector about the plane perpendicular to $|\omega\rangle$ (marked item)
- The diffusion reflects about $|s\rangle$
- Two reflections = rotation by 2θ where $\sin(\theta) = 1/\sqrt{N}$

- After k Grover iterations: angle rotates by $2k\theta$ toward $|\omega\rangle$

Initial angle from $|\omega\rangle$: $\theta \approx 1/\sqrt{N}$ (small, since marked item is rare)

Target angle: $\pi/2$ (perpendicular to $|s\rangle$, parallel to $|\omega\rangle$)

Rotations needed: $\pi/2 / (2\theta) \approx \pi\sqrt{N}/4 \approx \pi/4 \cdot \sqrt{N}$

Optimal number of iterations: $\sim(\pi/4)\sqrt{N}$

Why Over-Rotating Is Bad

After 0 iterations: probability of finding target = $1/N$ (uniform)

After k iterations: amplitude of target $\approx \sin((2k+1)\theta)$

After $(\pi/4)\sqrt{N}$ iters: probability ≈ 1 (maximum)

After $(\pi/2)\sqrt{N}$ iters: probability ≈ 0 (rotated past target!)

After $(3\pi/4)\sqrt{N}$ iters: probability ≈ 1 again

...

This oscillatory behavior is unique to quantum search. A classical algorithm's success probability only increases monotonically.

Code Breakdown: Grover's Algorithm in Qiskit

```

from qiskit import QuantumCircuit
import numpy as np

def grover_oracle(n, target):
    """
    Creates an oracle that marks the target state with a phase flip.
    n: number of qubits
    target: integer (the marked item)
    """
    oracle = QuantumCircuit(n)

    # Convert target to binary string (big-endian)
    target_bin = format(target, f'0{n}b')

    # Apply X gates to flip 0-bits (so all qubits are |1> for target state)
    for i, bit in enumerate(reversed(target_bin)):
        if bit == '0':
            oracle.x(i)    # Flip qubits that should be 0 in target

    # Multi-controlled Z gate: applies phase flip only when all qubits are |1>
    oracle.h(n-1)          # H before CNOT makes it a controlled-Z
    oracle.mcx(list(range(n-1)), n-1) # Multi-controlled X on last qubit
    oracle.h(n-1)          # H after: CZ → controlled-phase-flip

    # Uncompute: reapply X gates to restore
    for i, bit in enumerate(reversed(target_bin)):
        if bit == '0':
            oracle.x(i)

    return oracle

def diffusion_operator(n):
    """
    Grover's diffusion operator:  $2|s\rangle\langle s| - I$ 
    Reflects amplitudes about the uniform superposition.
    """
    diffusion = QuantumCircuit(n)

    # Apply  $H^n$ 
    diffusion.h(range(n))

```



```

# Apply  $X^n$ 
diffusion.x(range(n))

# Multi-controlled Z (phase flip on  $|0\dots 0\rangle$ )
diffusion.h(n-1)
diffusion.mcx(list(range(n-1)), n-1)
diffusion.h(n-1)

# Uncompute
diffusion.x(range(n))
diffusion.h(range(n))

return diffusion

def grover_circuit(n, target):
    """
    Complete Grover's algorithm circuit.
    n: number of qubits (searches  $2^n$  items)
    target: the marked item (integer 0 to  $2^n-1$ )
    """
    N = 2**n
    # Optimal iterations
    num_iterations = int(np.round(np.pi / 4 * np.sqrt(N)))

    qc = QuantumCircuit(n, n)

    # Step 1: Initialize uniform superposition
    qc.h(range(n)) #  $H^n|0\dots 0\rangle = (1/\sqrt{N})\sum|x\rangle$ 

    # Step 2: Grover iterations
    oracle = grover_oracle(n, target)
    diffusion = diffusion_operator(n)

    for _ in range(num_iterations):
        qc.compose(oracle, inplace=True) # Oracle: marks target
        qc.compose(diffusion, inplace=True) # Diffusion: amplifies target

    # Step 3: Measure
    qc.measure(range(n), range(n))

    return qc, num_iterations

```

```
# Example: 3 qubits, 8 items, target = 5 (binary: 101)
n = 3
target = 5
circuit, iters = grover_circuit(n, target)
print(f"Searching {2**n} items for target {target}")
print(f"Optimal iterations: {iters}")
print(circuit.draw())
```

Running the circuit:

```
from qiskit_aer import AerSimulator
from qiskit import transpile
from qiskit.visualization import plot_histogram

simulator = AerSimulator()
compiled = transpile(circuit, simulator)
job = simulator.run(compiled, shots=1024)
counts = job.result().get_counts()

# Expected: target '101' dominates
print(counts) # e.g., {'101': 891, '000': 18, '001': 20, ...}

plot_histogram(counts)
# Should show clear peak at binary representation of target=5
```

Internal Working: Geometric Picture

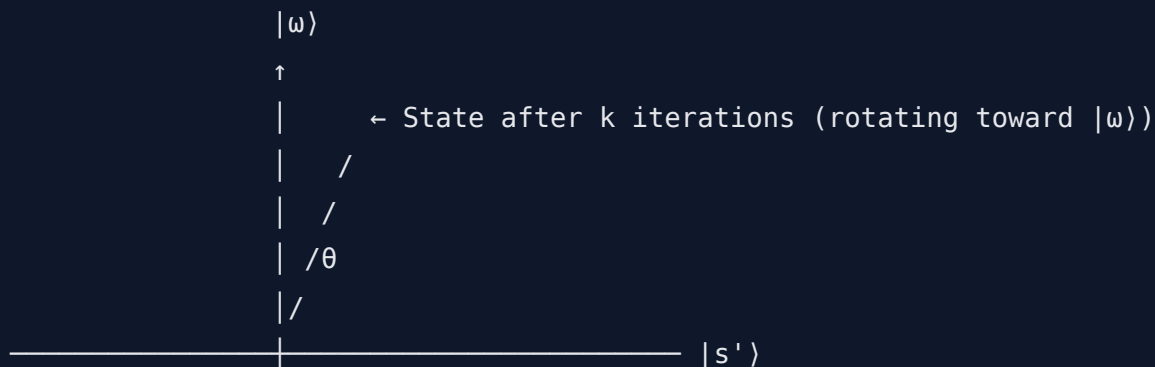
Grover's algorithm can be understood as rotation in a 2D subspace:

2D subspace spanned by:

$|\omega\rangle = |\text{target}\rangle$ (the marked item)

$|s'\rangle = (1/\sqrt{N-1}) \sum_{x \neq \omega} |x\rangle$ (uniform over non-marked items)

Initial state: $|s\rangle \approx (1/\sqrt{N})|\omega\rangle + \sqrt{(N-1)/N}|s'\rangle$



Oracle: Reflects about $|s'\rangle$ (flips $|\omega\rangle$ component sign)

Diffusion: Reflects about $|s\rangle$ (initial uniform superposition)

Net effect: Rotation by 2θ toward $|\omega\rangle$ per Grover iteration

Step-by-Step Execution: 2-Qubit Example (N=4, target=|11>)

$N = 4$, $n = 2$ qubits, target = $|11\rangle$, optimal iterations = 1

Initial: $|00\rangle|00\rangle$

Step 1: $H^{\otimes 2}$: $|s\rangle = (1/2)(|00\rangle + |01\rangle + |10\rangle + |11\rangle)$

Amplitudes: $[1/2, 1/2, 1/2, 1/2]$

Target probability: $1/4 = 25\%$

Step 2: Oracle (phase flip on $|11\rangle$):

Amplitudes: $[1/2, 1/2, 1/2, -1/2]$

Average amplitude: $(1/2 + 1/2 + 1/2 - 1/2)/4 = 1/4$

Target probability: still 25% (phase $\neq$ probability)

Step 3: Diffusion (reflect about average):

New amplitude of $|x\rangle = 2 \cdot \text{average} - \text{old_amplitude}$

$|00\rangle$: $2 \cdot (1/4) - 1/2 = 0$

$|01\rangle$: $2 \cdot (1/4) - 1/2 = 0$

$|10\rangle$: $2 \cdot (1/4) - 1/2 = 0$

$|11\rangle$: $2 \cdot (1/4) - (-1/2) = 1$

Amplitudes: $[0, 0, 0, 1]$

Target probability: 100%!

Step 4: Measure $\rightarrow |11\rangle$ with certainty.

For $N=4$, exactly 1 iteration is needed and gives 100% success probability.

Behind the Scenes: How Diffusion Works Mathematically

The diffusion operator $D = 2|s\rangle\langle s| - I$ can be decomposed:

$$D = H^{\otimes n} \cdot (2|0\dots 0\rangle\langle 0\dots 0| - I) \cdot H^{\otimes n}$$

Implementation:

$H^{\otimes n}$: Rotate to computational basis

$X^{\otimes n}$: Flip all qubits

MCZ: Multi-controlled-Z (phase flip $|1\dots 1\rangle = \text{flips } |0\dots 0\rangle$ after $X^{\otimes n}$)

$X^{\otimes n}$: Uncompute flip

$H^{\otimes n}$: Rotate back

This implements the reflection $2|s\rangle\langle s| - I$ about the uniform superposition.

Real World Usage

- **Cryptography:** Grover's algorithm weakens symmetric key cryptography (AES-128 becomes AES-64 effective security). NIST responded by recommending AES-256 as post-quantum secure.
- **Optimization:** Grover can be used as a subroutine to search solution spaces in optimization problems
- **Database search:** Not directly applicable to classical databases (requires quantum RAM), but valuable for searching quantum data
- **IBM Research:** Grover demonstrated on IBM 5-qubit device; increasingly used in algorithm benchmarking

Common Bugs

Bug	Cause	Fix
All outputs equally likely	Oracle not applied correctly	Verify oracle flips only target state
Target appears but with low probability	Too few or too many iterations	Calculate $k = \text{round}(\pi/4 \cdot \sqrt{N})$
qasm_simulator different from real device	Real noise	Use noise model simulation first
Negative result for large N	Grover not the right algorithm	Grover only searches; needs oracle implementation

Interview Questions

Basic:

- What is the search problem that Grover's algorithm solves?
- What is the speedup achieved by Grover's algorithm?
- What does the oracle do in Grover's algorithm?

Intermediate:

- Explain the diffusion operator and why it amplifies the marked item's amplitude.
- What happens if you run too many Grover iterations?
- Why is Grover's speedup quadratic and not exponential?

Advanced:

- Prove that Grover's $O(\sqrt{N})$ query complexity is optimal — no quantum algorithm can do better.
- How does Grover's algorithm affect the security of AES-128?

- Explain amplitude amplification as a generalization of Grover's algorithm.

Key Takeaways

- Grover's algorithm solves unstructured search in $O(\sqrt{N})$ queries vs. classical $O(N)$
- The algorithm works by: Initialize $\rightarrow$ (Oracle + Diffusion) $\times O(\sqrt{N}) \rightarrow$ Measure
- Oracle marks target with a phase flip; Diffusion amplifies marked amplitude
- Over-iterating rotates past the target — optimal iterations $\approx (\pi/4)\sqrt{N}$
- Quadratic speedup: important for cryptography (weakens symmetric ciphers), optimization subroutines
- Proved to be optimal — no quantum algorithm can search faster than Grover

WEEK 4 — QUANTUM ERROR CORRECTION AND NISQ ALGORITHMS

Lectures 17–19 — Quantum Error Correction

Objective

Understand why quantum computers suffer from errors, learn the theory of quantum error correction (QEC), study the three-qubit bit-flip code and the Shor code, and understand the threshold theorem that defines the path to fault-tolerant quantum computing.

Big Picture

```
Physical Qubits (noisy)
  ↓
Error Correction Code (encodes 1 logical qubit in many physical qubits)
  ↓
Syndrome Measurement (detects errors without measuring the quantum state)
  ↓
Classical Decoding (determines what error occurred)
  ↓
Correction (apply inverse of error)
  ↓
Protected Logical Qubit (effectively noise-free if error rate below threshold)
  ↓
Fault-Tolerant Quantum Computation
```

Core Concepts

Why Quantum Errors Are Hard to Correct

Challenge 1: Continuous errors. Classical bits can only flip (0→1). Quantum errors can be any rotation on the Bloch sphere — infinitely many possible errors. This seems to make QEC impossible.

Challenge 2: No-cloning. We can't copy quantum states to compare them classically.

Challenge 3: Measurement destroys. Looking at a qubit to check for errors collapses it.

The resolution (discovered independently by Shor and Steane in 1995):

- Error discretization: QEC codes correct for continuous errors by projecting onto discrete error syndromes
- Syndrome measurement: Detects errors without revealing (or disturbing) the logical state
- Entanglement: Used to encode one logical qubit across many physical qubits

Types of Quantum Errors

Error Type	Effect on Bloch Sphere	Matrix
Bit flip (X error)	Rotation by π around X-axis	Pauli X
Phase flip (Z error)	Rotation by π around Z-axis	Pauli Z
Bit + Phase flip (Y error)	Rotation by π around Y-axis	Pauli Y
Depolarizing	Random X, Y, or Z with equal probability	Mixed
Amplitude damping		$ 1\rangle \rightarrow$
Phase damping	Loss of phase coherence (T2)	Non-unitary

Key insight: Any single-qubit error can be expressed as a linear combination of I, X, Y, Z (the Pauli matrices form a basis). If we can correct X errors and Z errors independently, we can correct ANY error on that qubit.

Three-Qubit Bit-Flip Code

Idea: Encode one logical qubit into 3 physical qubits:

```
|0>_L = |000>
|1>_L = |111>

General logical qubit:  $\alpha|0>_L + \beta|1>_L = \alpha|000> + \beta|111>$ 
```

Encoding circuit:

```

qc = QuantumCircuit(3)
# Start with logical qubit on qubit 0
qc.cx(0, 1) # CNOT: copies bit pattern to qubit 1
qc.cx(0, 2) # CNOT: copies bit pattern to qubit 2
# Now:  $|\psi\rangle|00\rangle \rightarrow \alpha|000\rangle + \beta|111\rangle$ 

```

Error detection (syndrome measurement):

Measure parity of pairs without measuring individual qubits:

```

Syndrome qubits (ancilla):
  Parity(q0,q1) → ancilla bit 1
  Parity(q1,q2) → ancilla bit 2

qc.cx(0, 3) # Ancilla 3 XORed with qubit 0
qc.cx(1, 3) # Ancilla 3 XORed with qubit 1 → measures XOR(q0,q1)
qc.cx(1, 4) # Ancilla 4 XORed with qubit 1
qc.cx(2, 4) # Ancilla 4 XORed with qubit 2 → measures XOR(q1,q2)

```

Syndrome table:

Syndrome ($q0 \oplus q1, q1 \oplus q2$)	Error Detected
00	No error
11	Error on qubit 0 → apply X(0)
10	Error on qubit 1 → apply X(1)
01	Error on qubit 2 → apply X(2)

Why syndrome doesn't disturb logical state:

The syndrome measures PARITY of qubits, not their individual values. For the codeword $\alpha|000\rangle + \beta|111\rangle$:

- $\text{Parity}(q0,q1) = 0$ in both $|000\rangle$ and $|111\rangle \rightarrow$ syndrome gives 0 without distinguishing α from β
- After one qubit flips (say $|001\rangle$), parity changes $\rightarrow$ syndrome reveals error location

Limitation: The three-qubit code only corrects bit-flip (X) errors, not phase-flip (Z) errors.

Three-Qubit Phase-Flip Code

By symmetry, work in the Hadamard basis:

$$|0\rangle_L = |+++ \rangle = (|+\rangle)^{\otimes 3}$$

$$|1\rangle_L = |--- \rangle = (|-\rangle)^{\otimes 3}$$

Corrects Z errors in the same way the bit-flip code corrects X errors.

Shor's Nine-Qubit Code (First Complete QEC Code)

Peter Shor combined the bit-flip and phase-flip codes:

$$|0\rangle_L = (1/2\sqrt{2})(|000\rangle + |111\rangle)^{\otimes 3} \quad (\text{three copies of 3-qubit encoding})$$

$$|1\rangle_L = (1/2\sqrt{2})(|000\rangle - |111\rangle)^{\otimes 3}$$

- Uses 9 physical qubits to encode 1 logical qubit
- Corrects ANY single-qubit error (bit-flip, phase-flip, or any rotation)
- First proof that quantum error correction is possible

The Threshold Theorem

Statement: If the error rate per physical gate is below a threshold ϵ_{th} (typically around 10^{-3} to 10^{-2} , depending on the code), then arbitrarily long quantum computations can be performed using fault-tolerant quantum error correction with only polynomial overhead.

Physical error rate $< \epsilon_{th}$ (e.g., $10^{-3} = 0.1\%$)

↓

Concatenate QEC codes (encode qubits in qubits)

↓

Effective logical error rate decreases exponentially with concatenation levels

↓

Arbitrarily reliable quantum computation

Current IBM hardware: Error rates per two-qubit gate $\approx 0.1\text{--}1\%$. The threshold is $\approx 0.1\%$ for the surface code. IBM devices are approaching threshold but fault-tolerant computation requires more progress.

The surface code: Today's most promising QEC code, requires ~ 1000 physical qubits per logical qubit at current error rates. IBM's roadmap aims for this.

Stabilizer Formalism

QEC codes are efficiently described using **stabilizers** — operators that leave codewords invariant (+1 eigenvalue):

Three-qubit code stabilizers:

$Z_0 Z_1$ (measures parity of q_0, q_1) $\rightarrow +1$ for $|000\rangle$ and $|111\rangle$

$Z_1 Z_2$ (measures parity of q_1, q_2) $\rightarrow +1$ for $|000\rangle$ and $|111\rangle$

An X error on q_0 :

$Z_0 Z_1 (X_0 |\psi\rangle) = Z_0 X_0 Z_1 |\psi\rangle = -X_0 Z_0 Z_1 |\psi\rangle = -X_0 (+1 |\psi\rangle) = -|\psi\rangle$

$\rightarrow$ Syndrome gives $-1 \rightarrow$ error detected on qubit 0

Visual Mental Models

Error Correction Flow

Alice (logical state $\alpha|0\rangle_L + \beta|1\rangle_L$)

|

| Encoding: 1 logical qubit $\rightarrow$ n physical qubits

|

▼

Physical qubits travel through noisy channel

|

| Errors: X, Z, Y on individual physical qubits

▼

Syndrome measurement (ancilla qubits)

|

| Reveals which physical qubit has error

| Does NOT reveal logical state

▼

Classical decoder processes syndrome

|

▼

Apply correction to physical qubits

|

▼

Decode: n physical qubits $\rightarrow$ 1 logical qubit

|

▼

Bob receives $\alpha|0\rangle_L + \beta|1\rangle_L$ (error-free)

Encoding Rate of QEC Codes

Code	Physical	Logical	Errors corrected
-----	-----	-----	-----
3-qubit	3	1	1 bit-flip
3-qubit	3	1	1 phase-flip
Shor	9	1	1 arbitrary error
Steane	7	1	1 arbitrary error
Surface	$\sim 10^3$	1	~ 10 arbitrary errors (threshold regime)

Common Bugs

Mistake	Reality
"We can just check qubits for errors"	Measurement collapses the state — syndrome measurement uses ancilla qubits
"Shor's code copies the state 3 times"	It encodes (not copies) into entangled 9-qubit state
"Below threshold means zero errors"	Errors still happen; error correction suppresses them exponentially

Interview Questions

Basic:

- Why is quantum error correction harder than classical?
- What is a bit-flip error and how is it detected in the three-qubit code?
- Why doesn't syndrome measurement collapse the logical state?

Intermediate:

- Explain the Shor nine-qubit code and what errors it corrects.
- State the threshold theorem and explain its significance.
- What is the surface code and why is it the leading candidate for fault-tolerant QC?

Advanced:

- Explain the stabilizer formalism. How do stabilizers define a QEC code?
- Why is the threshold $\approx 10^{-3}$ and what determines it?

Key Takeaways

- Quantum errors are continuous but discretize under measurement (Pauli error basis)
- Syndrome measurement detects errors without disturbing the logical state

- Three-qubit code corrects bit-flip OR phase-flip; Shor's 9-qubit code corrects both
- Threshold theorem: below a critical error rate, arbitrarily reliable quantum computation is possible
- Fault-tolerant quantum computing requires ~1000 physical qubits per logical qubit — the major engineering challenge of the 2020s–2030s

Lectures 20–22 — NISQ Era: Variational Quantum Eigensolver (VQE)

Objective

Understand the Variational Quantum Eigensolver (VQE) — the flagship NISQ-era quantum algorithm for finding the ground state energy of molecules. Learn the hybrid quantum-classical optimization loop that makes VQE viable on noisy hardware.

Big Picture

```
Chemistry problem: Find lowest energy (ground state) of a molecule
    ↓
Map molecular Hamiltonian to qubit operators (Jordan-Wigner or BK transformation)
    ↓
Prepare parameterized trial state  $|\psi(\theta)\rangle$  on quantum computer
    ↓
Measure expectation value  $\langle\psi(\theta)|H|\psi(\theta)\rangle$  on quantum computer
    ↓
Classical optimizer updates parameters  $\theta$  to minimize energy
    ↓
Repeat until convergence → Ground state energy estimate
```

Core Concepts

The Variational Principle

The **variational principle** from quantum mechanics states:

For any trial state $|\psi\rangle$:

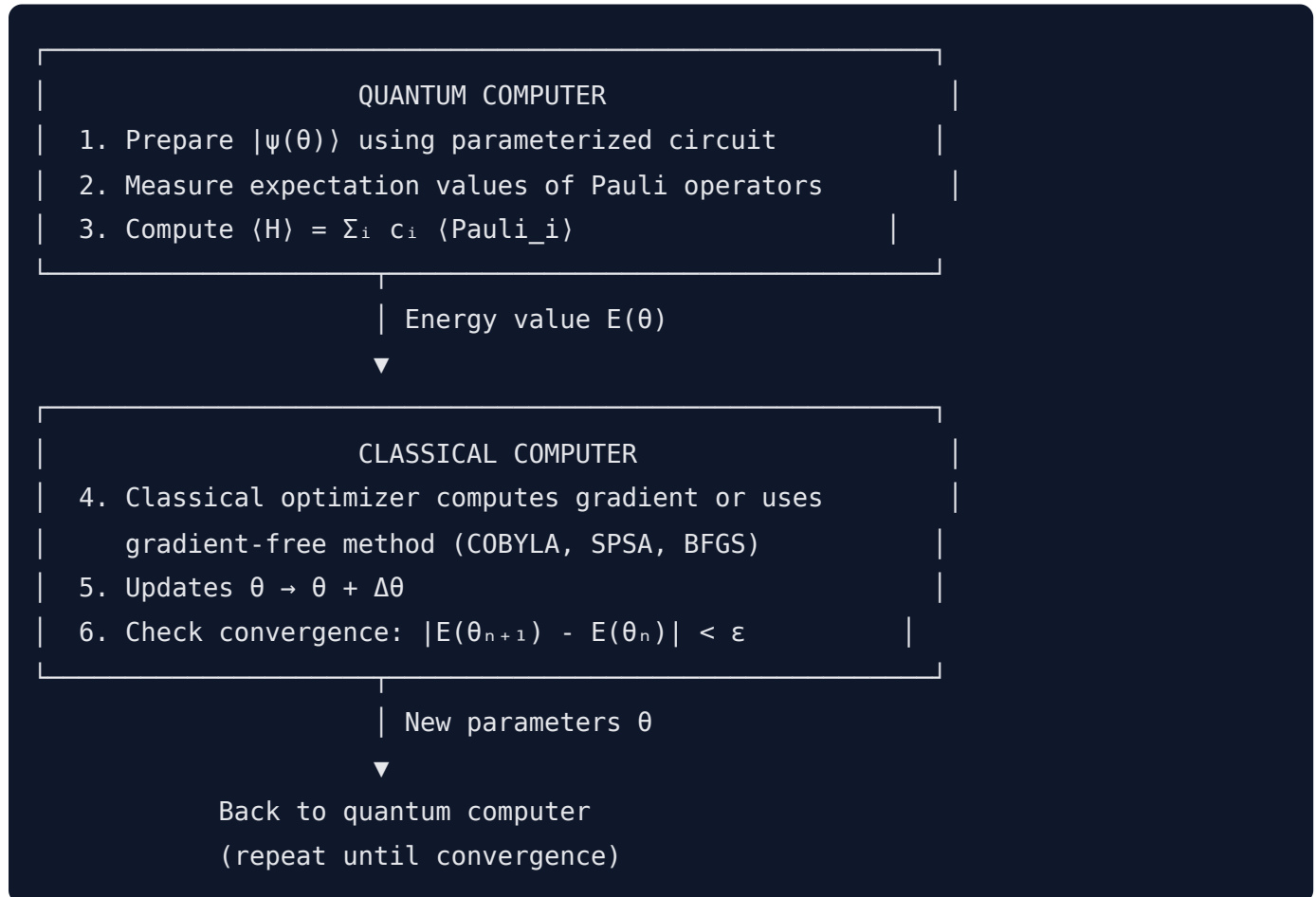
$$\langle\psi|H|\psi\rangle \geq E_0$$

Where E_0 is the true ground state energy.

This means if we search over all possible trial states $|\psi(\theta)\rangle$ (parameterized by θ) and minimize $\langle\psi(\theta)|H|\psi(\theta)\rangle$, we get the best approximation to the ground state energy.

VQE turns this into an optimization problem solvable by a hybrid quantum-classical loop.

The VQE Loop



Mapping Chemistry to Qubits

Step 1: Second quantization

Represent the molecular Hamiltonian using fermionic creation/annihilation operators acting on molecular orbital occupations.

Step 2: Jordan-Wigner or Bravyi-Kitaev transformation

Map fermionic operators to qubit (Pauli) operators.

Example: Hydrogen molecule (H_2) Hamiltonian:

$$H = h_{00} \cdot Z_0 + h_{11} \cdot Z_1 + h_{01} \cdot Z_0 Z_1 + h_{01} \cdot X_0 X_1 + \dots$$

Where h_{ij} are molecular integrals computed classically and Z_0 , $X_0 X_1$, etc. are Pauli operators on qubits

H_2 requires only 2 qubits in a minimal basis — feasible even on NISQ devices!

Parameterized Quantum Circuits (Ansatz)

The trial state $|\psi(\theta)\rangle$ is prepared by a **parameterized quantum circuit** (also called an **ansatz**):

Hardware-efficient ansatz:

```
from qiskit.circuit.library import RealAmplitudes

ansatz = RealAmplitudes(num_qubits=2, reps=2)
# Creates: alternating RY gates + CNOT layers
# Parameters:  $[\theta_0, \theta_1, \theta_2, \theta_3, \theta_4, \theta_5, \theta_6, \theta_7]$ 
print(ansatz.draw())
```

Circuit structure:

```
q0: ─[Ry( $\theta_0$ )]─■──[Ry( $\theta_2$ )]─■──[Ry( $\theta_4$ )]─
           |           |
q1: ─[Ry( $\theta_1$ )]─x──[Ry( $\theta_3$ )]─x──[Ry( $\theta_5$ )]─
```

Chemically motivated ansatz (UCCSD):

The Unitary Coupled Cluster Singles and Doubles (UCCSD) ansatz is motivated by quantum chemistry. It encodes electronic excitations from a reference Hartree-Fock state.

VQE in Qiskit

```

from qiskit.circuit.library import RealAmplitudes
from qiskit.quantum_info import SparsePauliOp
from qiskit_algorithms import VQE
from qiskit_algorithms.optimizers import COBYLA
from qiskit_aer.primitives import Estimator

# Define H2 Hamiltonian (simplified)
# In real code, use qiskit_nature to compute these coefficients
hamiltonian = SparsePauliOp.from_list([
    ('ZI', -1.0523732),
    ('IZ', 0.39793742),
    ('ZZ', -0.39793742),
    ('XX', -0.01128010),
    ('YY', -0.01128010)
])

# Define ansatz
ansatz = RealAmplitudes(num_qubits=2, reps=1)

# Define classical optimizer
optimizer = COBYLA(maxiter=500)

# Create VQE instance
estimator = Estimator()
vqe = VQE(estimator, ansatz, optimizer)

# Run VQE
result = vqe.compute_minimum_eigenvalue(hamiltonian)
print(f'Ground state energy: {result.eigenvalue.real:.6f} Hartree')
# Expected for H2: approximately -1.137 Hartree

```

Challenges in VQE

Barren plateaus: In deep parameterized circuits, gradients vanish exponentially with the number of qubits, making optimization inefficient. Active research topic.

Noise sensitivity: On real hardware, noise corrupts the energy measurements. Error mitigation techniques (zero-noise extrapolation, symmetry verification) are needed.

Ansatz expressibility vs. trainability: More expressive ansätze can represent the ground state better but are harder to optimize (barren plateaus). Less expressive ansätze are easier to train but may not reach the true ground state.

Real World Usage

- **Merck KGaA:** Using VQE via IBM Quantum to simulate organic molecules for pharmaceutical design
- **Boeing:** VQE for materials simulation (lightweight high-strength materials)
- **JSR Corporation:** Quantum simulations for polymer chemistry
- **Total Energies:** Reservoir simulation for energy exploration
- **IBM Research:** Demonstrated VQE for BeH₂, H₂O, and other small molecules on real quantum hardware

Key Takeaways

- VQE = Variational Quantum Eigensolver = hybrid quantum-classical algorithm for quantum chemistry
- Uses variational principle: $\langle \psi(\theta) | H | \psi(\theta) \rangle \geq E_0$, minimize over θ
- Quantum computer prepares state and measures energy; classical optimizer updates parameters
- Most promising near-term application of quantum computing
- Challenges: barren plateaus, noise, ansatz design

Lectures 23–25 — NISQ Era: QAOA and Industrial Applications

Objective

Understand the Quantum Approximate Optimization Algorithm (QAOA), its application to combinatorial optimization problems, and survey how industry is deploying NISQ quantum algorithms today.

Big Picture

```
Combinatorial optimization problem (e.g., Max-Cut, portfolio optimization)
    ↓
Encode as Ising/QUBO (Quadratic Unconstrained Binary Optimization) Hamiltonian
    ↓
QAOA circuit: alternating problem and mixer unitaries
    ↓
Classical optimization of QAOA parameters
    ↓
Measure best solution found
```

Core Concepts

The Optimization Problem: Max-Cut

Max-Cut problem: Given a graph $G = (V, E)$ with vertices V and edges E , partition vertices into two sets S and $\bar{S}$ to maximize the number of edges between the sets (cut edges).

Why it matters:

- Social network community detection
- VLSI circuit design
- Financial portfolio optimization (reformulated)
- Supply chain optimization
- NP-hard in general: no efficient classical algorithm known for large instances

Example:

```
Graph:    1 -- 2
          |    |
          4 -- 3
```

Best cut: $\{1,3\}$ vs $\{2,4\}$

Cut edges: $(1,2), (2,3), (3,4), (1,4) = 4$ edges

(This is the maximum cut = 4 = all edges for a cycle graph)

Encoding Max-Cut as a Quantum Problem

Assign qubit $z_i \in \{+1, -1\}$ to vertex i . Edge (i,j) is cut if $z_i \neq z_j$, i.e., $z_i \cdot z_j = -1$.

Max-Cut objective:

$$\text{Maximize } C = \sum_{(i,j) \in E} (1 - z_i z_j) / 2$$

In Pauli notation (mapping $z \rightarrow Z$ Pauli matrix):

$$C = \sum_{(i,j) \in E} (I - Z_i Z_j) / 2$$

This is a Hamiltonian! Maximizing C = minimizing $-C$ = finding ground state of $-C$ Hamiltonian.

QAOA Circuit

QAOA prepares a trial state using alternating layers of:

1. **Problem unitary** $U_p(\gamma)$: Encodes the objective function
2. **Mixer unitary** $U_m(\beta)$: Drives transitions between solution states

$U_p(\gamma) = e^{-i\gamma C} = \text{product of ZZ rotations over edges}$
 $U_m(\beta) = e^{-i\beta B} = \text{product of X rotations over all vertices}$

The QAOA circuit of depth p :

$$|\psi(\gamma, \beta)\rangle = U_m(\beta_p)U_p(\gamma_p)\dots U_m(\beta_1)U_p(\gamma_1)|s\rangle$$

Where $|s\rangle = H^{\otimes n}|0\dots 0\rangle = \text{uniform superposition}$
 $\gamma = [\gamma_1, \dots, \gamma_p]$, $\beta = [\beta_1, \dots, \beta_p]$ are parameters

Classical optimizer maximizes: $\langle \psi(\gamma, \beta) | C | \psi(\gamma, \beta) \rangle$

As $p \rightarrow \infty$, QAOA approaches the exact solution (adiabatic limit). For small p , QAOA gives approximate solutions.

QAOA in Qiskit

```

from qiskit.circuit.library import QAOAAnsatz
from qiskit.quantum_info import SparsePauliOp
from qiskit_algorithms import QAOA
from qiskit_algorithms.optimizers import COBYLA
from qiskit_aer.primitives import Sampler

# Max-Cut on 4-node cycle graph: edges (0,1),(1,2),(2,3),(3,0)
# Hamiltonian: C = (1/2)Σ_{edges} (I - ZiZj)
edges = [(0, 1), (1, 2), (2, 3), (3, 0)]
pauli_list = []
for i, j in edges:
    pauli_str = ['I'] * 4
    pauli_str[i] = 'Z'
    pauli_str[j] = 'Z'
    pauli_list.append(''.join(reversed(pauli_str)), -0.5)
    pauli_list.append('IIII', 0.5)

hamiltonian = SparsePauliOp.from_list(pauli_list)

# QAOA with p=2 layers
sampler = Sampler()
optimizer = COBYLA(maxiter=300)
qaoa = QAOA(sampler, optimizer, reps=2)

result = qaoa.compute_minimum_eigenvalue(hamiltonian)
print(f'Best cut value: {-result.eigenvalue.real:.4f}')
print(f'Best solution: {result.best_measurement}')

```

Industrial NISQ Applications

Company	Application	Algorithm	Hardware
Volkswagen	Traffic flow optimization (Lisbon)	Quantum annealing	D-Wave
Goldman Sachs	Derivative pricing, risk analysis	QAE (Amplitude Estimation)	IBM
JPMorgan Chase	Portfolio optimization	QAOA	IBM/IonQ
Airbus	Aircraft loading optimization	VQE/QAOA	Various
BMW	Supply chain optimization	QAOA	IBM
Merck	Drug molecule simulation	VQE	IBM

ExxonMobil	Fluid simulation	Variational algorithms	Various
Daimler	Battery simulation (Lithium-Air)	VQE	IBM

The Promise and Reality of NISQ

Optimistic view: NISQ devices with 50–1000 qubits can already do things impossible to simulate classically for specific tasks. VQE and QAOA offer genuine near-term value.

Cautious view: Most NISQ applications can currently be matched by classical heuristics (simulated annealing, genetic algorithms). Quantum advantage in practice has not been demonstrated for practically relevant optimization problems.

The honest assessment: We are in a period of experimentation. Some near-term quantum applications will prove valuable; many will not. The field is progressing rapidly. This course gives you the foundation to participate in figuring out which is which.

Interview Questions

Basic:

- What is QAOA and what type of problems does it solve?
- Explain the Max-Cut problem.
- What are the two unitaries that alternate in QAOA?

Intermediate:

- How does QAOA relate to adiabatic quantum computation?
- What is the QAOA parameter landscape and what optimization challenges arise?
- Compare VQE and QAOA: problem domains, circuit structure, classical optimization.

Advanced:

- For what p (circuit depth) does QAOA guarantee optimal solutions?
- Explain why classical simulation of QAOA circuits becomes hard as n grows.
- What is the quantum advantage (if any) of QAOA over classical approximation algorithms like the Goemans-Williamson SDP?

Key Takeaways

- QAOA alternates problem Hamiltonian and mixing Hamiltonian unitaries to search solution space
- Directly applicable to Max-Cut, TSP, portfolio optimization, scheduling
- Parameterized by γ and β vectors; classical optimizer finds good parameters
- Near-term quantum algorithms (VQE, QAOA) use hybrid quantum-classical loops to work within NISQ constraints

- Genuine industrial quantum computing is early-stage but real — major companies investing heavily

Lecture 26 — Invited Talk: Future of Quantum Computing and Quantum Error Correction (IBM Perspective)

Objective

Synthesize the entire course with an industry perspective on where quantum computing is heading, IBM's hardware roadmap, the path from NISQ to fault-tolerant quantum computing, and career opportunities in quantum.

Core Concepts

IBM Quantum Roadmap

```
2021: Eagle (127 qubits) — First >100 qubit device
2022: Osprey (433 qubits) — Improved connectivity
2023: Condor (1121 qubits) — First 1000+ qubit device
2024: Heron (133 qubits) — Higher quality, lower error rate
2025: Flamingo (156 qubits per module, multi-chip)
2026+: Error-corrected logical qubits (target)
2030+: Fault-tolerant utility-scale quantum computing
```

IBM's strategy prioritizes **quality** (error rate) over raw qubit count. A 100-qubit device with 0.01% error rates is more useful than a 1000-qubit device at 1% error rates.

The Path to Fault-Tolerance

```
Current: NISQ (50-1000 physical qubits, ~0.1-1% errors)
      ↓ Better hardware + error mitigation
Near-term: Utility-scale NISQ (1000-10000 physical qubits, ~0.01% errors)
      ↓ Implement QEC codes
Medium-term: Early fault-tolerant (100 logical qubits, surface code)
      ↓ More qubits, better decoding
Long-term: Full fault-tolerant (1000+ logical qubits, break RSA-2048)
```

The crossover point — where quantum computers genuinely outperform classical for practically relevant problems — is called **quantum advantage** or **quantum utility**. IBM's 2023 paper in Nature demonstrated quantum utility on a specific physics problem (kicked Ising model).

Career Opportunities

Academic path:

- Quantum algorithm research (new algorithms, complexity theory)
- Quantum error correction (codes, decoders, threshold analysis)
- Quantum hardware (physics, engineering, materials science)
- Quantum chemistry / simulation (applications)

Industry path:

- Quantum software engineer (Qiskit, circuit optimization)
- Quantum applications scientist (VQE/QAOA for industry problems)
- Quantum systems engineer (hardware calibration, control)
- Quantum business development (identifying industrial use cases)

Skills to develop:

1. Linear algebra and quantum mechanics fundamentals
2. Python + Qiskit proficiency
3. One domain expertise (chemistry, optimization, ML, cryptography)
4. Understanding of error correction and noise
5. Classical algorithms for comparison and benchmarking

Key Takeaways

- IBM's roadmap: quality over quantity; targeting early fault-tolerance in the 2025-2030 window
- Quantum utility is being demonstrated now on specific problems; general utility is a decade away
- The quantum workforce gap is real — now is the time to build skills
- This course provides the foundation for a career in quantum: fundamentals → algorithms → hardware → applications

End of Playlist Summary

Complete Concept Map

QUANTUM COMPUTING: CONCEPT MAP

LINEAR ALGEBRA (Foundation)

- ├ Complex vector spaces
- ├ Bra-ket notation
- ├ Inner products, tensor products
- └ Unitary matrices

|



QUANTUM MECHANICS

- ├ Superposition
- ├ Measurement (probabilistic, destructive)
- ├ Entanglement
- ├ Phase and interference
- └ Unitary evolution

|



QUBIT AND GATES

- ├ Single-qubit gates (H, X, Y, Z, Rx, Ry, Rz)
- ├ Two-qubit gates (CNOT, CZ, SWAP)
- ├ Multi-qubit gates (Toffoli, MCX)
- └ Universal gate sets

|



QUANTUM PROTOCOLS

- ├ Quantum teleportation
- ├ Superdense coding
- └ No-cloning theorem

|



ALGORITHMS	ERROR CORRECTION	HARDWARE/SDK
Oracle model	Error types	IBM Quantum
Phase kickback	3-qubit bit-flip	Qiskit
Deutsch-Jozsa	3-qubit phase-flip	Circuit Composer
Grover's search	Shor 9-qubit code	Real backends
VQE	Stabilizer formalism	Transpilation
QAOA	Threshold theorem	Noise models

|

|

|

```
graph TD; A[ ] --> B[NISQ APPLICATIONS]; B --> C[FAULT-TOLERANT QUANTUM COMPUTING (future)];
```

NISQ APPLICATIONS

- └─ Quantum chemistry (VQE → drug design)
- └─ Optimization (QAOA → logistics, finance)
- └─ Quantum machine learning
- └─ Quantum cryptography (QKD, PQC)

FAULT-TOLERANT QUANTUM COMPUTING (future)

- └─ Shor's algorithm (cryptography)
- └─ Large-scale chemistry simulation
- └─ Quantum AI

Learning Timeline

Week 0 (Pre-course): Linear algebra review

→ Vectors, matrices, eigenvalues, complex numbers

Week 1 (Lectures 1-4): Foundations

→ Motivation: why quantum? India's quantum mission. Applications.

→ Mathematics: qubits, superposition, measurement, entanglement, Dirac notation

Week 2 (Lectures 5-10): Tools and Protocols

→ IBM Quantum platform: Composer + Qiskit

→ Quantum gates: H, X, Y, Z, CNOT, rotations

→ Bell states, no-cloning theorem, quantum teleportation

→ First hands-on circuits

Week 3 (Lectures 11-16): Algorithms

→ Oracle model: quantum black boxes and phase kickback

→ Deutsch-Jozsa: first exponential quantum speedup (1 query vs. 2^n)

→ Grover's algorithm: quadratic speedup for search ($O(\sqrt{N})$ vs. $O(N)$)

→ Full Qiskit implementation and hardware runs

Week 4 (Lectures 17-26): Applied and Advanced

→ Quantum error correction: bit-flip code, Shor code, threshold theorem

→ VQE: hybrid quantum-classical for quantum chemistry

→ QAOA: hybrid optimization for combinatorial problems

→ Industry applications and future roadmap

Technology Stack

Technology	Role	Version/Notes
Python	Programming language for Qiskit	3.8+
Qiskit	IBM's quantum SDK	1.x (significant API changes from 0.x)
qiskit-aer	Quantum circuit simulator	Supports statevector, QASM, noise
qiskit-ibm-runtime	Cloud access to IBM quantum hardware	Replaces qiskit-ibmq-provider
qiskit-algorithms	VQE, QAOA implementations	Replaces qiskit.algorithms

numpy	Numerical computation (state vectors, matrices)	1.21+
scipy	Classical optimization routines (BFGS, etc.)	1.7+
matplotlib	Visualization of results	3.4+
IBM Quantum Composer	Web browser-based visual circuit builder	No install needed
IBM Quantum Lab	Cloud Jupyter notebooks	No install needed
Jupyter Notebook	Local interactive Python environment	Optional

Cheat Sheet

Qubit State

$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$ where $|\alpha|^2 + |\beta|^2 = 1$
 $P(\text{measuring } 0) = |\alpha|^2$, $P(\text{measuring } 1) = |\beta|^2$

Key Single-Qubit Gates

X (NOT): $|0\rangle \rightarrow |1\rangle$, $|1\rangle \rightarrow |0\rangle$
 H: $|0\rangle \rightarrow |+\rangle$, $|1\rangle \rightarrow |-\rangle$ (creates superposition)
 Z: $|0\rangle \rightarrow |0\rangle$, $|1\rangle \rightarrow -|1\rangle$ (phase flip)
 T: $|0\rangle \rightarrow |0\rangle$, $|1\rangle \rightarrow e^{i\pi/4}|1\rangle$

Key Two-Qubit Gates

CNOT: $|00\rangle \rightarrow |00\rangle$, $|01\rangle \rightarrow |01\rangle$, $|10\rangle \rightarrow |11\rangle$, $|11\rangle \rightarrow |10\rangle$
 (flips target when control= $|1\rangle$)

Bell States

$|\Phi^+\rangle = (|00\rangle + |11\rangle)/\sqrt{2}$ $|\Phi^-\rangle = (|00\rangle - |11\rangle)/\sqrt{2}$
 $|\Psi^+\rangle = (|01\rangle + |10\rangle)/\sqrt{2}$ $|\Psi^-\rangle = (|01\rangle - |10\rangle)/\sqrt{2}$

Phase Kickback

$$U_f|x\rangle|- \rangle = (-1)^{f(x)}|x\rangle|- \rangle$$

Algorithm Complexities

Unstructured search: Classical $O(N)$, Grover $O(\sqrt{N})$
Factoring: Classical $O(\exp)$, Shor $O(\text{poly})$
Deutsch-Jozsa: Classical $O(2^n)$, DJ $O(1)$

VQE One-Liner

Minimize $\langle \psi(\theta) | H | \psi(\theta) \rangle$ over θ using quantum computer + classical optimizer

QAOA One-Liner

Alternating problem ($e^{-i\gamma C}$) and mixer ($e^{-i\beta B}$) unitaries, optimize γ, β classically

Qiskit Minimal Example

```
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
qc = QuantumCircuit(2, 2)
qc.h(0); qc.cx(0,1); qc.measure([0,1],[0,1])
sim = AerSimulator()
counts = sim.run(qc).result().get_counts()
```

Glossary

Term	Definition
Qubit	Quantum bit; a two-level quantum system that can be in a superposition of
Superposition	A quantum system existing in multiple states simultaneously with complex probability amplitudes
Entanglement	Quantum correlation between two or more qubits that cannot be described by independent single-qubit states

Bra-ket notation	Dirac's notation: ket $ \psi\rangle$ for column vectors, bra $\langle\psi $ for row vectors (conjugate transpose)
Unitary operator	Matrix U satisfying $U^\dagger U = I$; describes reversible quantum evolution
Hadamard gate (H)	Creates equal superposition from
CNOT gate	Controlled-NOT; two-qubit entangling gate that flips target qubit when control qubit is
Bell state	One of four maximally entangled two-qubit states; canonical entangled states
No-cloning theorem	Fundamental quantum theorem: an unknown quantum state cannot be perfectly copied
Quantum teleportation	Protocol to transfer a qubit's state using entanglement + 2 classical bits
Quantum oracle	Black-box unitary U_f that evaluates $f(x)$: $U_f x\rangle y\rangle = x\rangle y \oplus f(x)\rangle$
Phase kickback	Technique where $U_f x\rangle -\rangle = (-1)^{f(x)} x\rangle -\rangle$ encodes $f(x)$ into phase
Deutsch-Jozsa algorithm	Determines if a function is constant or balanced in 1 oracle query (vs. $2^{(n-1)+1}$ classically)
Grover's algorithm	Searches N items for a marked element in $O(\sqrt{N})$ queries (vs. $O(N)$ classically)
Amplitude amplification	Generalization of Grover's: amplifies amplitude of "good" states in any quantum algorithm
Bloch sphere	Geometric representation of a single qubit state as a point on a unit sphere
Measurement	Quantum operation that collapses a quantum state to a classical outcome probabilistically
Decoherence	Loss of quantum state due to interaction with environment; limits quantum computation
T1 (amplitude damping)	Time constant for qubit to decay from
T2 (dephasing)	Time constant for qubit to lose phase coherence between
Transpilation	Process of compiling a quantum circuit for specific hardware: gate decomposition + qubit routing
NISQ	Noisy Intermediate-Scale Quantum; current era of 50-1000 noisy physical qubits

VQE	Variational Quantum Eigensolver; hybrid quantum-classical algorithm for ground state energy
QAOA	Quantum Approximate Optimization Algorithm; hybrid algorithm for combinatorial optimization
Ansatz	Parameterized trial quantum state/circuit used in variational algorithms
Barren plateau	Region in parameterized circuit landscape where gradients vanish exponentially; training pitfall
Quantum error correction (QEC)	Encoding logical qubits into multiple physical qubits to detect and correct errors
Bit-flip error	Pauli X error; flips
Phase-flip error	Pauli Z error; negates amplitude of
Syndrome measurement	Ancilla-based measurement that reveals error location without disturbing logical state
Stabilizer	Operator that leaves all codeword states invariant (+1 eigenvalue); defines QEC code
Threshold theorem	If physical error rate < threshold, fault-tolerant quantum computation is possible with polynomial overhead
Surface code	Leading QEC code for 2D hardware; requires ~1000 physical qubits per logical qubit near threshold
Qiskit	IBM's open-source Python quantum computing SDK
QuantumCircuit	Core Qiskit object representing a quantum circuit with gates, registers, and measurements
Shots	Number of times a quantum circuit is executed to sample the output probability distribution
Jordan-Wigner transformation	Maps fermionic operators to qubit (Pauli) operators for quantum chemistry simulation
Hamiltonian	Operator representing total energy of a quantum system; central object in quantum chemistry
Ground state	Lowest energy eigenstate of a Hamiltonian; target of VQE
Variational principle	$\langle \psi H \psi \rangle \geq E_0$ for any state $ \psi\rangle$; basis of VQE

Max-Cut	Graph optimization problem: partition vertices to maximize crossing edges; solved by QAOA
QUBO	Quadratic Unconstrained Binary Optimization; standard form for quantum optimization problems
Post-quantum cryptography	Classical cryptographic algorithms designed to be secure against quantum attacks
QKD	Quantum Key Distribution; provably secure key exchange using quantum mechanics

Common Interview Topics (Full Playlist)

Fundamentals

1. What is a qubit and how does it differ from a classical bit?
2. Write the mathematical representation of a general qubit state.
3. What is quantum superposition and how is it different from classical probability?
4. Define quantum entanglement formally. How is the Bell state $|\Phi^+\rangle = (|00\rangle + |11\rangle)/\sqrt{2}$ entangled?
5. What happens during quantum measurement? (collapse, probabilistic outcomes)
6. Why must quantum gates be unitary operations?
7. Explain global phase vs. relative phase and why only the latter matters physically.

Gates and Circuits

8. What does the Hadamard gate do? Write its matrix.
9. What is the CNOT gate? When does it flip the target qubit?
10. How do H and CNOT together create entanglement?
11. What is a universal gate set? Give an example.
12. What is transpilation and why is it needed for real quantum hardware?
13. Why are IBM's native gates {ID, RZ, SX, X, CX} and not just H and CNOT?

Algorithms

14. State the Deutsch-Jozsa problem and describe the algorithm.
15. Explain phase kickback and why it is useful.
16. How does Grover's algorithm work? What are the oracle and diffusion operator?
17. Why is Grover's speedup quadratic, not exponential?
18. What is the optimal number of Grover iterations? What happens if you overshoot?
19. How does Grover's algorithm impact the security of AES-128?

Error Correction

20. Why is quantum error correction harder than classical?
21. Explain the three-qubit bit-flip code. What errors does it correct?
22. What is a syndrome measurement and how does it work without collapsing the logical state?
23. State the threshold theorem and explain its significance.
24. Why does fault-tolerant quantum computing require so many physical qubits?

NISQ Algorithms

25. Explain VQE: what problem does it solve, what is the variational principle?
26. Describe the VQE hybrid quantum-classical loop.
27. What is QAOA and what type of problems does it address?
28. Compare VQE and QAOA: when would you use each?
29. What is a barren plateau and why is it a problem for variational algorithms?
30. Name three real companies using quantum computing today and their applications.

Quantum Applications

31. Why does Shor's algorithm threaten RSA cryptography?
 32. What is quantum key distribution (QKD) and how does it provide security?
 33. Why is quantum simulation of molecules advantageous over classical simulation?
 34. What is the current state of quantum advantage: where has it been demonstrated?
 35. Describe India's NM-QTA: scope, funding, and goals.
-

Mini Project Ideas

Beginner

1. **Bell State Explorer:** Build a Qiskit circuit that creates all four Bell states. Visualize their measurement statistics. Verify entanglement by comparing correlated outcomes.
2. **Gate Truth Table Generator:** For each single-qubit gate (X, Y, Z, H, T, S), compute the output for $|0\rangle$ and $|1\rangle$ inputs. Display as a circuit diagram and output state.
3. **Bloch Sphere Animator:** Using Qiskit visualization, apply a sequence of gates to a qubit and visualize the Bloch sphere state at each step.
4. **Quantum Coin Flipper:** Build a quantum random number generator using superposition. Compare outcome statistics with a classical random number generator.

Intermediate

5. **Deutsch-Jozsa Solver:** Implement Deutsch-Jozsa for $n=1,2,3,4$ bits with multiple constant and balanced oracle types. Benchmark against a classical randomized algorithm.
6. **Grover's Search Benchmark:** Implement Grover's for $n=2,3,4,5$ qubits. Plot success probability vs. number of iterations. Compare to theoretical prediction ($\sin^2((2k+1)\theta)$).
7. **Quantum Teleportation Circuit:** Full teleportation protocol in Qiskit with arbitrary input state. Verify that Bob's qubit matches Alice's original state using state fidelity.
8. **Noise Impact Study:** Run the Bell state circuit on a real IBM backend and the simulator. Compare histograms. Quantify the error introduced by real hardware noise.

Advanced

9. **VQE for H₂:** Implement VQE for the hydrogen molecule using Qiskit Nature. Compare with exact classical diagonalization. Plot energy vs. bond length (potential energy surface).
10. **QAOA for Max-Cut:** Implement QAOA for various graph topologies (cycle, complete graph, random). Study how approximation ratio varies with QAOA depth p .
11. **QEC Simulation:** Simulate the three-qubit bit-flip code with injected X errors. Verify that syndrome measurement correctly identifies error locations.
12. **Grover's Satisfiability Solver:** Encode a 3-SAT problem as a Grover oracle. Demonstrate finding a satisfying assignment using quantum search.

Industry-Level

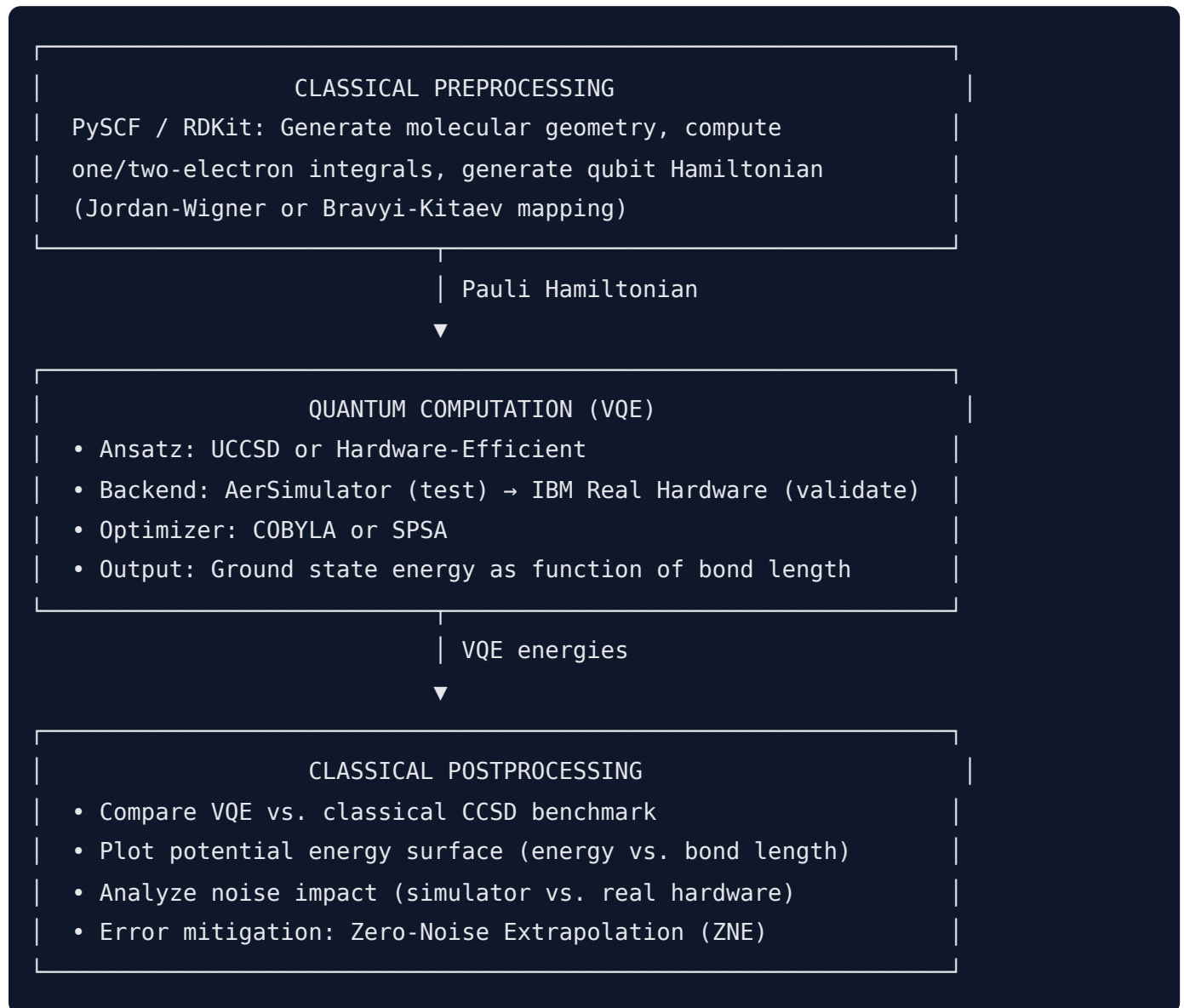
13. **Portfolio Optimization with QAOA:** Formulate a portfolio optimization problem (N stocks, risk/return constraints) as QUBO. Apply QAOA and compare with classical mean-variance optimization.
14. **Drug Molecule VQE Pipeline:** Using Qiskit Nature + PySCF, compute the ground state energy of LiH or BeH₂. Compare VQE results with CCSD classical baseline. Analyze convergence and noise sensitivity.
15. **Quantum Key Distribution Simulator:** Simulate the BB84 QKD protocol. Include eavesdropper attacks and demonstrate that eavesdropping is detectable through error rate increase.

Capstone Project

Project: Quantum Molecular Property Predictor

Overview: Build an end-to-end pipeline that takes a small molecule (H₂, LiH, or BeH₂), computes its ground state energy and other properties using VQE on IBM Quantum, and compares results with classical quantum chemistry methods.

Architecture:



Folder Structure:

```
quantum-molecular-predictor/
|
├─ README.md
├─ requirements.txt          # qiskit, qiskit-nature, pyscf, matplotlib
|
├─ data/
|   └─ molecules/           # Molecular geometry files (.xyz)
|
├─ src/
|   ├── molecule_setup.py    # Generate Hamiltonian from molecular geometry
|   ├── ansatz_builder.py    # UCCSD and hardware-efficient ansatz
|   ├── vqe_runner.py        # VQE main loop (simulator + real hardware)
|   ├── error_mitigation.py  # ZNE and symmetry verification
|   └─ visualizer.py         # Plots: PES, convergence, noise comparison
|
├─ notebooks/
|   ├── 01_Introduction.ipynb
|   ├── 02_Hamiltonian_Mapping.ipynb
|   ├── 03_VQE_Simulator.ipynb
|   ├── 04_VQE_Real_Hardware.ipynb
|   └─ 05_Analysis_and_Results.ipynb
|
└─ results/
    ├── h2_pes_simulation.png
    ├── h2_pes_hardware.png
    └─ convergence_plots/
```

Tech Stack:

Layer	Technology
Quantum SDK	Qiskit 1.x
Chemistry	Qiskit Nature + PySCF
Optimization	SciPy (COBYLA, L-BFGS-B)
Visualization	Matplotlib, Seaborn
Error mitigation	Qiskit IBM Runtime (ZNE)
Classical benchmark	PySCF (FCI, CCSD)

Features:

1. **Molecule input:** Accept any small molecule geometry file
 2. **Hamiltonian generation:** JW and BK mappings with qubit count comparison
 3. **Ansatz selection:** UCCSD (chemistry-motivated) vs. Hardware-efficient (NISQ-practical)
 4. **Backend flexibility:** Simulation → Real IBM hardware with automatic queue management
 5. **Convergence tracking:** Plot VQE energy vs. optimizer iterations
 6. **Noise analysis:** Compare noiseless simulation vs. real hardware vs. error-mitigated results
 7. **Potential energy surface:** Sweep bond lengths and plot E vs. r
 8. **Classical comparison:** Benchmark VQE against CCSD and FCI classical methods
-

Learning Objectives:

By completing this project, you will:

- Build a complete quantum chemistry application from scratch
 - Understand the end-to-end pipeline: chemistry → math → circuit → hardware → result
 - Experience the challenges of real quantum hardware (noise, queue, transpilation)
 - Apply error mitigation techniques to improve real-hardware results
 - Develop intuition for where quantum computing provides genuine value vs. where classical is still better
 - Gain portfolio-quality experience with Qiskit and quantum algorithms
-

Extensions:

- Add support for larger molecules using quantum phase estimation (QPE) instead of VQE
 - Implement custom error mitigation (symmetry verification, probabilistic error cancellation)
 - Compare multiple classical optimizers (COBYLA, SPSA, gradient descent with parameter shift rule)
 - Extend to excited states using quantum EOM-VQE
 - Deploy as a web application using FastAPI + quantum backend in the cloud
-

Further Learning

Books

Book	Authors	Level	Focus
------	---------	-------	-------

Quantum Computation and Quantum Information	Nielsen & Chuang	Graduate	Complete reference; the "bible" of QC
Quantum Computing: An Applied Approach	Hidary	Intermediate	Practical, Qiskit examples
An Introduction to Quantum Computing	Kaye, Laflamme, Mosca	Undergraduate	Gentle introduction, CS-oriented
Quantum Computer Science	Mermin	Undergraduate	Very accessible for CS students
Quantum Algorithms via Linear Algebra	Lipton & Regan	Undergraduate	Linear algebra first approach

Official Documentation

- **Qiskit Documentation:** <https://docs.quantum.ibm.com>
- **IBM Quantum Learning:** <https://learning.quantum.ibm.com> (free interactive courses)
- **Qiskit Textbook (legacy):** <https://qiskit.org/textbook/preface.html>
- **IBM Quantum Network:** <https://quantum.ibm.com/network>

Video Lectures and MOOCs

- **MIT OpenCourseWare 8.370:** Quantum Computation (Peter Shor) — rigorous CS/physics treatment
- **edX: Quantum Computing Fundamentals (MIT)** — accessible introduction
- **Coursera: Introduction to Quantum Computing (Saint Petersburg State University)** — algorithm focus
- **YouTube: 3Blue1Brown** — Linear algebra visual intuition (prerequisite foundation)
- **YouTube: IBM Quantum Learning Series** — Official IBM tutorials:
<https://www.youtube.com/playlist?list=PLOFEBzvs-Vvp2xg9-POLJhQwtVktLYGbY>

Research Papers (Landmark)

Paper	Significance
Shor (1994): "Algorithms for Quantum Computation"	Shor's factoring algorithm
Grover (1996): "A Fast Quantum Mechanical Algorithm for Database Search"	Grover's search
Shor (1995): "Scheme for reducing decoherence in quantum computer memory"	First QEC code
Steane (1996): "Error Correcting Codes in Quantum Theory"	Steane 7-qubit code

Preskill (2018): "Quantum Computing in the NISQ Era and Beyond"	Coined "NISQ"
Peruzzo et al. (2014): "A variational eigenvalue solver on a photonic quantum processor"	First VQE
Farhi et al. (2014): "A Quantum Approximate Optimization Algorithm"	Introduced QAOA
IBM Nature (2023): "Evidence for the utility of quantum computing before fault tolerance"	Quantum utility demonstration

GitHub Repositories

- **Qiskit:** <https://github.com/Qiskit/qiskit> — Main SDK
- **Qiskit Textbook source:** <https://github.com/Qiskit/textbook>
- **Qiskit Algorithms:** <https://github.com/qiskit-community/qiskit-algorithms> — VQE, QAOA implementations
- **Qiskit Nature:** <https://github.com/qiskit-community/qiskit-nature> — Quantum chemistry
- **Mitig:** <https://github.com/unitaryfund/mitiq> — Error mitigation library
- **PennyLane:** <https://github.com/PennyLaneAI/pennylane> — Alternative QML-focused SDK

Advanced Topics to Learn Next

1. **Shor's Algorithm:** Full treatment of quantum Fourier transform + period finding → integer factorization
2. **Quantum Phase Estimation (QPE):** General algorithm for eigenvalue estimation; used in Shor's and chemistry
3. **HHL Algorithm:** Quantum linear systems algorithm (with important caveats)
4. **Quantum Error Mitigation:** Zero-noise extrapolation, probabilistic error cancellation, symmetry verification
5. **Surface Code:** The leading path to fault-tolerant quantum computing
6. **Quantum Machine Learning:** QNNs, quantum kernels, quantum GANs
7. **Advanced Variational Algorithms:** ADAPT-VQE, quantum EOM-VQE, MA-QAOA
8. **Quantum Networks:** Quantum repeaters, entanglement distribution, quantum internet
9. **Post-Quantum Cryptography:** CRYSTALS-Kyber, CRYSTALS-Dilithium, SPHINCS+
10. **Adiabatic Quantum Computing:** D-Wave, quantum annealing, AQC-to-circuit equivalence

This handbook was compiled from the NPTEL IIT Madras–IBM course "Introduction to Quantum Computing: Quantum Algorithms and Qiskit" (Course ID: 106106232), instructed by Prof. Prabha

Mandayam (IIT Madras), Dr. Anupama Ray (IBM Research), and Dr. Sheshashayee Raghunathan (IBM Systems). It is intended as a comprehensive study companion and revision resource.